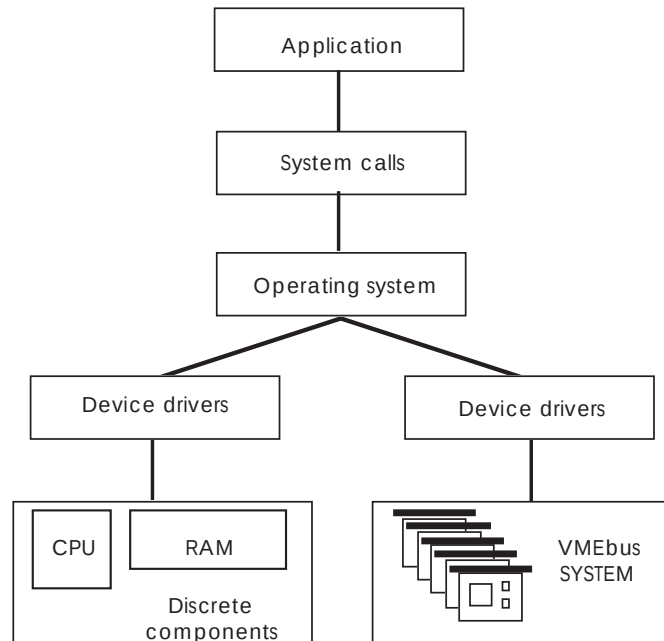


7

Real-time operating systems

What are operating systems?

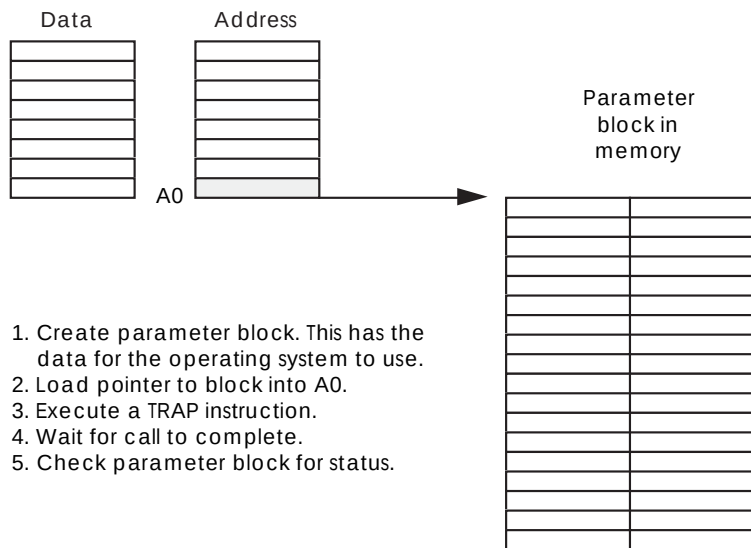
Operating systems are software environments that provide a buffer between the user and the low level interfaces to the hardware within a system. They provide a constant interface and a set of utilities to enable users to utilise the system quickly and efficiently. They allow software to be moved from one system to another and therefore can make application programs hardware independent. Program debugging tools are usually included which speed up the testing process. Many applications do not require any operating system support at all and run direct on the hardware.



Hardware independence through the use of an operating system

Such software includes its own I/O routines, for example, to drive serial and parallel ports. However, with the addition of mass storage and the complexities of disk access and file structures, most applications immediately delegate these tasks to an operating system.

The delegation decreases software development time by providing system calls to enable application software access to any of the I/O system facilities. These calls are typically made by building a parameter block, loading a specified register with its location and then executing a software interrupt instruction.



Typical system call mechanism for the M680x0 processor family

The TRAP instruction is the MC68000 family equivalent of the software interrupt and switches the processor into supervisor mode to execute the required function. It effectively provides a communication path between the application and the operating system kernel. The kernel is the heart of the operating system which controls the hardware and deals with interrupts, memory usage, I/O systems etc. It locates a parameter block by using an address pointer stored in a predetermined address register. It takes the commands stored in a parameter block and executes them. In doing so, it is the kernel that drives the hardware, interpreting the commands passed to it through a parameter block. After the command is completed, status information is written back into the parameter block, and the kernel passes control back to the application which continues running in USER mode. The application will find the I/O function completed with the data and status information written into the parameter block. The application has had no direct access to the memory or hardware whatsoever.

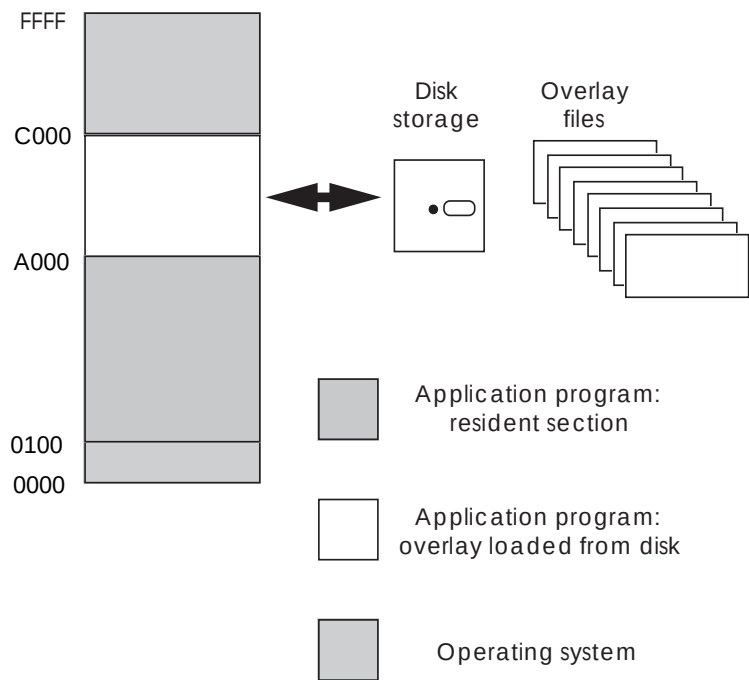
These parameter blocks are standard throughout the operating system and are not dependent on the actual hardware performing the physical tasks. It does not matter if the system uses an MC68901 multifunction peripheral or a 8530 serial communication controller to provide the serial ports: the operating system driver software takes care of the dependencies. If the parameter blocks are general enough in their definition, data can be supplied from almost any source within the system, for example a COPY utility could use the same blocks to get data from a serial port and copy it to a parallel port, or for copying data from one file to another. This idea of device independence and unified I/O allows software to be reused rather than rewritten. Software can be easily moved from one system to another. This is important for modular

embedded designs, especially those that use an industry standard bus such as VMEbus, where system hardware can easily be upgraded and/or expanded.

Operating system internals

The first widely used operating system was CP/M, developed for the Intel 8080 microprocessor and 8" floppy disk systems. It supported I/O calls by two jump tables — BDOS (basic disk operating system) and BIOS (basic I/O system). It quickly became a standard within the industry and a large amount of application software became available for it. Many of the micro-based business machines of the late 1970s and early 1980s were based on CP/M. Its ideas even formed the basis of MS-DOS, chosen by IBM for its personal computers.

CP/M is a good example of a single tasking operating system. Only one task or application can be executed at any one time and therefore it only supports one user at a time. When an application is loaded, it provides the user-defined part of the total 'CP/M' program.



Program overlays

Any application program has to be complete and therefore the available memory often becomes the limiting factor. Program overlays are often used to solve this problem. Parts of the complete program are stored separately on disk and retrieved and loaded over an unused code area when needed. This allows applications larger than the available memory to run, but it places the control responsibility on the application. This is similar to virtual memory

schemes where the operating system divides a task's memory into pages and swaps them between memory and mass storage. However, the operating system assumes complete control and such schemes are totally transparent to the user.

With a single tasking operating system, it is not possible to run multiple tasks simultaneously. Large applications have to be run sequentially and cannot support concurrent operations. There is no support for message passing or task control, which would enable applications to be divided into separate entities. If a system needs to take log data and store it on disk and, at the same time, allow a user to process that data using an online database package, a single tasking operating system would need everything to be integrated. With a multitasking operating system, the data logging task can run at the same time as the database. Data can be passed between each element by a common file on disk, and neither task need have any direct knowledge of the other. With a single tasking system, it is likely that the database program would have to be written from scratch. With the multitasking system, a commercial program can be used, and the logging software interfaced to it. These restrictions forced many applications to interface directly with the hardware and therefore lose the hardware independence that the operating system offered. Such software would need extensive modification to port it to another configuration.

Multitasking operating systems

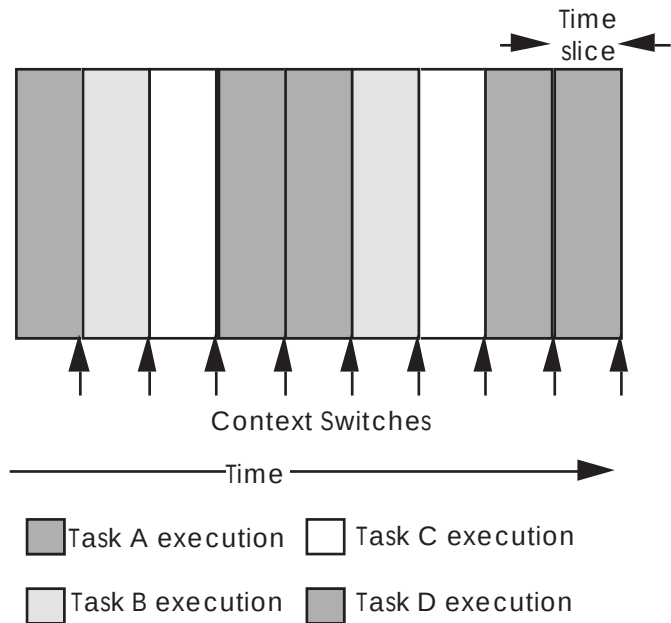
For the majority of embedded systems, a single tasking operating system is too restrictive. What is required is an operating system that can run multiple applications simultaneously and provide intertask control and communication. The facilities once only available to mini and mainframe computer users are now required by 16/32 bit microprocessor users.

A multitasking operating system works by dividing the processor's time into discrete time slots. Each application or task requires a certain number of time slots to complete its execution. The operating system kernel decides which task can have the next slot, so instead of a task executing continuously until completion, its execution is interleaved with other tasks. This sharing of processor time between tasks gives the illusion to each user that he is the only one using the system.

Context switching, task tables, and kernels

Multitasking operating systems are based around a multitasking kernel which controls the time slicing mechanisms. A time slice is the time period each task has for execution before it is stopped and replaced during a context switch. This is periodically triggered by a hardware interrupt from the system timer. This interrupt may provide the system clock and several interrupts may be executed and counted before a context switch is performed.

When a context switch is performed, the current task is interrupted, the processor’s registers are saved in a special table for that particular task and the task is placed back on the ‘ready’ list to await another time slice. Special tables, often called task control blocks, store all the information the system requires about the task, for example its memory usage, its priority level within the system and its error handling. It is this context information that is switched when one task is replaced by another.



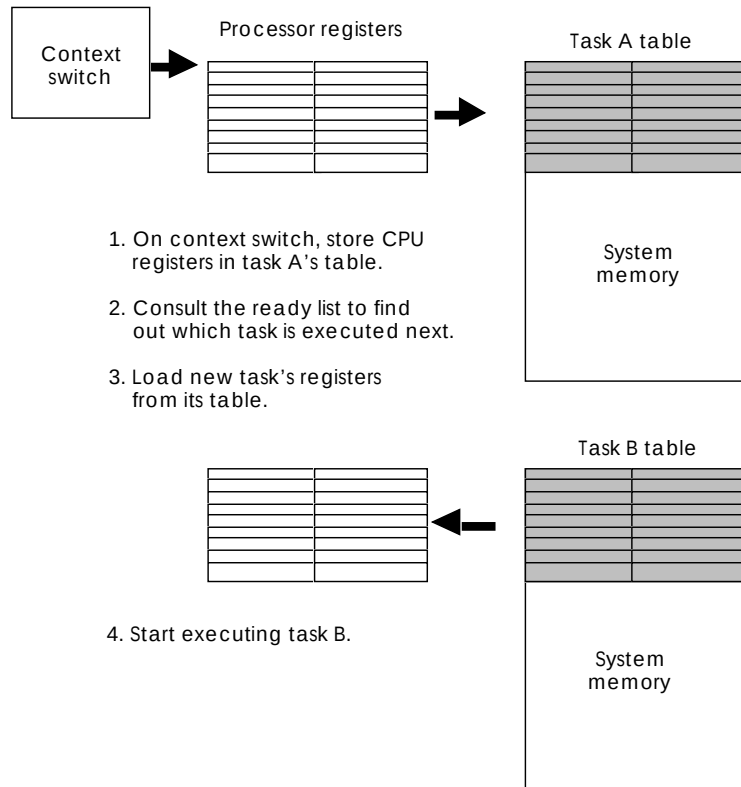
Time slice mechanism for multitasking operating systems

The ‘ready’ list contains all the tasks and their status and is used by the scheduler to decide which task is allocated the next time slice. The scheduling algorithm determines the sequence and takes into account a task’s priority and present status. If a task is waiting for an I/O call to complete, it will be held in limbo until the call is complete.

Once a task is selected, the processor registers and status at the time of its last context switch are loaded back into the processor and the processor is started. The new task carries on as if nothing had happened until the next context switch takes place. This is the basic method behind all multitasking operating systems.

The diagram shows a simplified state diagram for a typical real-time operating system which uses this time slice mechanism. On each context switch, a task is selected by the kernel’s scheduler from the ‘ready’ list and is put into the run state. It is then executed until another context switch occurs. This is normally signalled by a periodic interrupt from a timer. In such cases the task is simply switched out and put back on the ‘ready’ list, awaiting its next slot. Alternatively, the execution can be stopped by the task executing certain kernel commands. It could suspend itself, where it remains

present in the system but no further execution occurs. It could become dormant, awaiting a start command from another task, or even simply waiting for a server task within the operating system to perform a special function for it. A typical example of a server task is a driver performing special screen graphics functions. The most common reason for a task to come out of the run state, is to wait for a message or command, or delay itself for a certain time period. The various wait directives allow tasks to synchronise and control each other within the system. This state diagram is typical of many real-time operating systems.



Context switch mechanism

The kernel controls memory usage and prevents tasks from corrupting each other. If required, it also controls memory sharing between tasks, allowing them to share common program modules, such as high level language run-time libraries. A set of memory tables is maintained, which is used to decide if a request is accepted or rejected. This means that resources, such as physical memory and peripheral devices, can be protected from users without using hardware memory management provided the task is disciplined enough to use the operating system and not access the resources directly. This is essential to maintain the system's integrity.

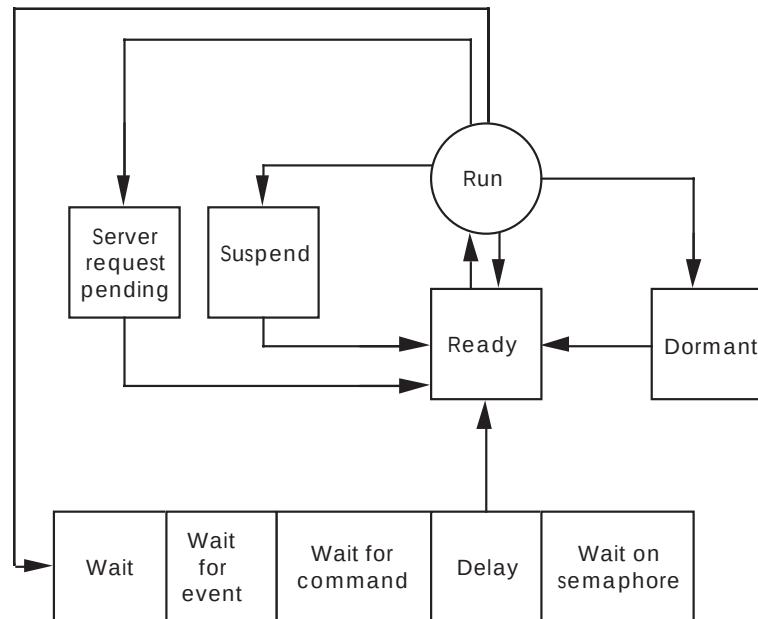
Message passing and control can be implemented in such systems by using the kernel to act as a message passer and

controller between tasks. If task A wants to stop task B, then by executing a call to the kernel, the status of task B can be changed and its execution halted. Alternatively, task B can be delayed for a set time period or forced to wait for a message.

With a typical real-time operating system, there are two basic type of messages that the kernel will deal with:

- flags that can control but cannot carry any implicit information — often called semaphores or events and
- messages which can carry information and control tasks — often called messages or events.

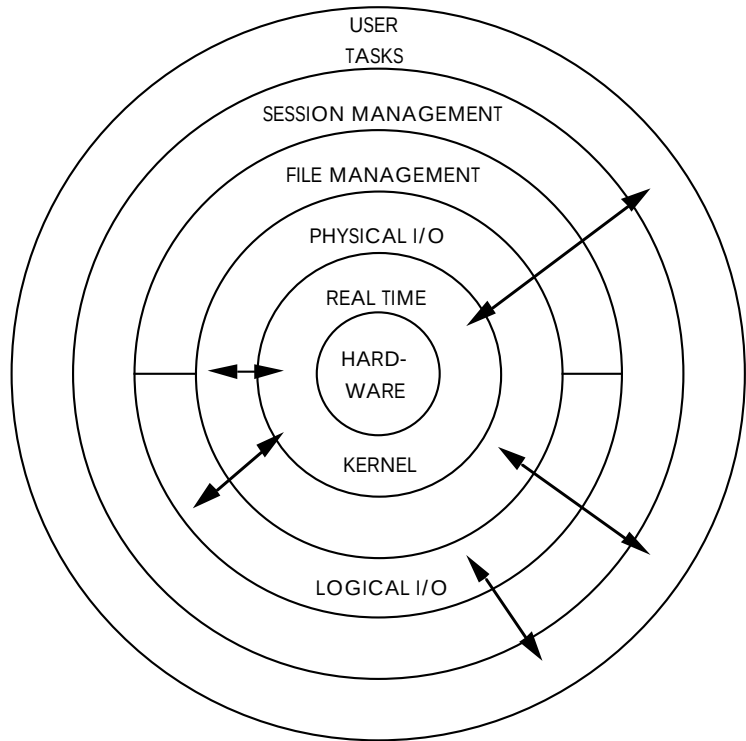
The kernel maintains the tables required to store this information and is responsible for ensuring that tasks are controlled and receive the information. With the facility for tasks to communicate between each other, system call support for accessing I/O, loading tasks from disk etc., can be achieved by running additional tasks, with a special system status. These system tasks provide additional facilities and can be included as required.



State diagram for a typical real-time kernel

To turn a real-time kernel into a full operating system with file systems and so on, requires the addition of several such tasks to perform I/O services, file handling and file management services, task loading, user interface and driver software. What was about a small <16 kbyte-sized kernel will often grow into a large 120 kbyte operating system. These extra facilities are built up as layers surrounding the kernel. Application tasks then fit around the outside. A typical onion structure is shown as an example. Due to the modular construction, applications can generally access any level directly if required. Therefore, application tasks that just

require services provided by the kernel can be developed and debugged under the full environment, and stripped down for integration onto the target hardware.



A typical operating system structure

In a typical system, all these service tasks and applications are controlled, scheduled and executed by the kernel. If an application wishes to write some data to a hard disk in the system, the process starts with the application creating a parameter block and asking the file manager to open the file. This system call is normally executed by a TRAP instruction. The kernel then places the task on its 'waiting' list until the file manager had finished and passed the status information back to the application task. Once this event has been received, it wakes up and is placed on the 'ready' list awaiting a time slot.

These actions are performed by the kernel. The next application command requests the file handling services to assign an identifier — often called a logical unit number (LUN) — to the file prior to the actual access. This is needed later for the I/O services call. Again, another parameter block is created and the file handler is requested to assign the LUN. The calling task is placed on the 'waiting' list until this request is completed and the LUN returned by the file handler. The LUN identifies a particular I/O resource such as a serial port or a file without actually knowing its physical characteristics. The device is therefore described as logical rather than physical.

With the LUN, the task can create another parameter block, containing the data, and ask the I/O services to write the data to the file. This may require the I/O services to make system calls of its own. It may need to call the file services for more data or to pass further information on. The data is then supplied to the device driver which actually executes the instructions to physically write the data to the disk. It is generally at this level that the logical nature of the I/O request is translated into the physical characteristics associated with the hardware. This translation should lie in the domain of the device driver software. The user application is unaware of these characteristics.

A complex system call can cause many calls between the system tasks. A program loader that is requested by an application task to load another task from memory needs to call the file services and I/O services to obtain the file from disk, and the kernel to allocate memory for the task to be physically loaded.

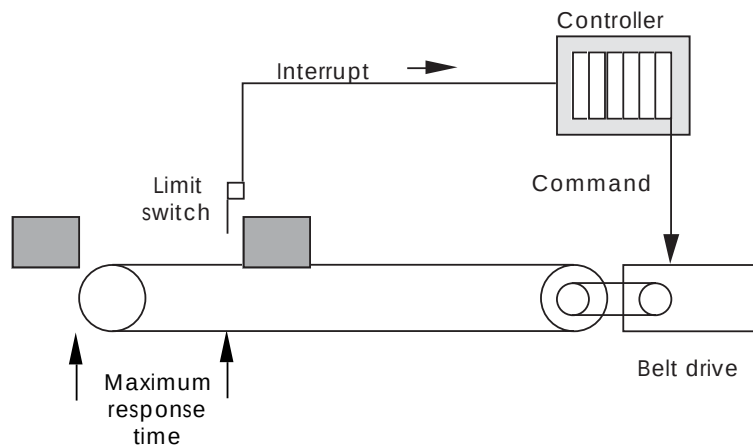
The technique of using standard names, files, and/or logical unit numbers to access system I/O makes the porting of application software from one system to another very easy. Such accesses are independent of the hardware the system is running on, and allow applications to treat data received or sent in the same way, irrespective of its source.

What is a real-time operating system?

Many multitasking operating systems available today are also described as 'real-time'. These operating systems provide additional facilities allowing applications that would normally interface directly with the microprocessor architecture to use interrupts and drive peripherals to do so without the operating system blocking such activities. Many multitasking operating systems prevent the user from accessing such sensitive resources. This overzealous caring can prevent many operating systems from being used in applications such as industrial control.

A characteristic of a real-time operating system is its defined response time to external stimuli. If a peripheral generates an interrupt, a real-time system will acknowledge and start to service it within a maximum defined time. Such response times vary from system to system, but the maximum time specified is a worst case figure, and will not be exceeded due to changes in factors such as system workload.

Any system meeting this requirement can be described as real-time, irrespective of the actual value, but typical industry accepted figures for context switches and interrupt response times are about 10 microseconds. This figure gets smaller as processors become more powerful and run at higher speeds. With several processors having the same context switch mechanism, the final context switch time come down to its clock speed and the memory access time.



Example of a real-time response

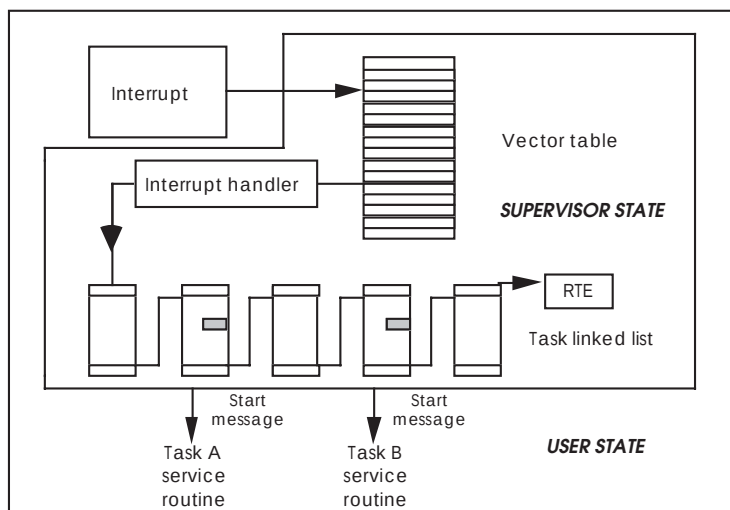
The consequences to industrial control of not having a real-time characteristic can be disastrous. If a system is controlling an automatic assembly line, and does not respond in time to a request from a conveyor belt limit switch to stop the belt, the results are easy to imagine. The response does not need to be instantaneous—if the limit switch is set so that there are 3 seconds to stop the belt, any system with a guaranteed worst case response of less than 3 seconds can meet this real-time requirement.

For an operating system to be real-time, its internal mechanisms need to show real-time characteristics so that the internal processes sequentially respond to external interrupts in guaranteed times.

When an interrupt is generated, the current task is interrupted to allow the kernel to acknowledge the interrupt and obtain the vector number that it needs to determine how to handle it. A typical technique is to use the kernel's interrupt handler to update a linked list which contains information on all the tasks that need to be notified of the interrupt.

If a task is attached to a vector used by the operating system, the system actions its own requirements prior to any further response by the task. The handler then sends an event message to the tasks attached to the vector, which may change their status and completely change the priorities of the task ready list. The scheduler analyses the list, and dispatches the highest priority task to run. If the interrupt and task priorities are high enough, this may be the next time slice.

The diagram depicts such a mechanism: the interrupt handler and linked list searches are performed by the kernel. The first priority is to service the interrupt. This may be from a disk controller indicating that it has completed a data transfer. Once the kernel has satisfied its own needs, the handler will start a linked list search. The list comprises blocks of data identifying tasks that have their own service routines. Each block will contain a reference to the next block, hence the linked list terminology.



Handling interrupt routines within an operating system

Each identified task is then sent a special message. This will start the task's service routine when it receives its next time slice. The kernel interrupt handler will finally execute an RTE return from the exception instruction which will restore the processor state prior to the interrupt. In such arrangements the task service routines execute in USER mode. The only SUPERVISOR operation is that of the kernel and its own interrupt handler. As can be imagined, this processing can increase the interrupt latency seen by the task quite dramatically. A ten-fold increase is not uncommon.

To be practical, a real-time operating system has to guarantee maximum response times for its interrupt handler, event passing mechanisms, scheduler algorithm and provide system calls to allow tasks to attach and handle interrupts.

With the conveyor belt example above, a typical software configuration would dedicate a task to controlling the conveyor belt. This task would make several system calls on start-up to access the parallel I/O peripheral that interfaces the system to components such as the drive motors and limit switches and tells the kernel that certain interrupt vectors are attached to the task and are handled by its own interrupt handling routine.

Once the task has set everything up, it remains dormant until an event is sent by other tasks to switch the belt on or off. If a limit switch is triggered, it sets off an interrupt which forces the kernel to handle it. The currently executing task stops, the kernel handler searches the task interrupt attachment linked list, and places the controller task on the ready list, with its own handler ready to execute. At the appropriate time slice, the handler runs, accesses the peripheral and switches off the belt. This result may not be normal, and so the task also sends event messages to the others, informing them that it has acted independently and may

force other actions. Once this has been done, the task goes back to its dormant state awaiting further commands.

Real-time operating systems have other advantages: to prevent a system from power failure usually needs a guaranteed response time so that the short time between the recognition of and the actual power failure can be used to store vital data and bring the system down in a controlled manner. Many operating systems actually have a power fail module built into the kernel so that no time is lost in executing the module code.

So far in this chapter, an overview of the basics behind a real-time operating system have been explained. There are, however, several variants available for the key functions such as task swapping and so on. The next few sections will delve deeper into these topics.

Task swapping methods

The choice of scheduler algorithms can play an important part in the design of an embedded system and can dramatically affect the underlying design of the software. There are many different types of scheduler algorithm that can be used, each with either different characteristics or different approaches to solving the same problem of how to assign priorities to schedule tasks so that correct operation is assured.

Time slice

Time slicing has been previously mentioned in this chapter under the topic of multitasking and can be used within an embedded system where time critical operations are not essential. To be more accurate about its definition, it describes the task switching mechanism and not the algorithm behind it although its meaning has become synonymous with both.

Time slicing works by making the task switching regular periodic points in time. This means that any task that needs to run next will have to wait until the current time slice is completed or until the current task suspends its operation. This technique can also be used as a scheduling method as will be explained later in this chapter. The choice of which task to run next is determined by the scheduling algorithm and thus is nothing to do with the time slice mechanism itself. It just happens that many time slice-based systems use a round-robin or other fairness scheduler to distribute the time slices across all the tasks that need to run.

For real-time systems where speed is of the essence, the time slice period plus the context switch time of the processor determines the context switch time of the system. With most time slice periods in the order of milliseconds, it is the dominant factor in the system response. While the time period can be reduced to improve the system context switch time, it will increase the number of task switches that will occur and this will reduce the efficiency of the system. The larger the number of switches, the less time there is available for processing.

Pre-emption

The alternative to time slicing is to use pre-emption where a currently running task can be stopped and switched out — pre-empted — by a higher priority *active* task. The *active* qualifier is important as the example of pre-emption later in this section will show. The main difference is that the task switch does not need to wait for the end of a time slice and therefore the system context switch is now the same as the processor context switch.

As an example of how pre-emption works, consider a system with two tasks A and B. A is a high priority task that acts as an ISR to service a peripheral and is activated by a processor interrupt from the peripheral. While it is not servicing the peripheral, the task remains dormant and stays in a suspended state. Task B is a low priority task that performs system housekeeping.

When the interrupt is recognised by the processor, the operating system will process it and activate task A. This task with its higher priority compared to task B will cause task B to be pre-empted and replaced by task A. Task A will continue processing until it has completed and then suspend itself. At this point, task B will context switch task A out because task A is no longer active.

This can be done with a time slice mechanism provided the interrupt rate is less than the time slice rate. If it is higher, this can also be fine provided there is sufficient buffering available to store data without losing it while waiting for the next time slice point. The problem comes when the interrupt rate is higher or if there are multiple interrupts and associated tasks. In this case, multiple tasks may compete for the same time slice point and the ability to run even though the total processing time needed to run all of them may be considerably less than the time provided within a single time slot. This can be solved by artificially creating more context switch points by getting each task to suspend after completion. This may offer only a partial solution because a higher priority task may still have to wait on a lower priority task to complete. With time slicing, the lower priority task cannot be pre-empted and therefore the higher priority task must wait for the end of the time slice or the lower priority task to complete. This is a form of priority inversion which is explained in more detail later.

Most real-time operating systems support pre-emption in preference to time slicing although some can support both methodologies

Co-operative multitasking

This is the mechanism behind Windows 3.1 and while not applicable to real-time operating systems for reasons which will become apparent, it has been included for reference.

The idea of co-operative multitasking is that the tasks themselves co-operate between themselves to provide the illusion of multitasking. This is done by periodically allowing other tasks or applications the opportunity to execute. This requires program-

ming within the application and the system can be destroyed by a single rogue program that hogs all the processing power. This method may be acceptable for a desktop personal computer but it is not reliable enough for most real-time embedded systems.

Scheduler algorithms

So far in this section, the main methods of swapping tasks has been discussed. It is clear that pre-emption is the first choice for embedded systems because of its better system response. The next issue to address is how to assign the task priorities so that the system works and this is the topic that is examined now.

Rate monotonic

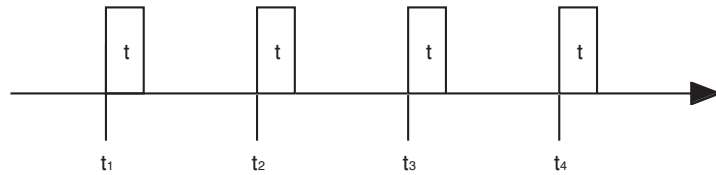
Rate monotonic scheduling (RMS) is an approach that is used to assign task priority for a pre-emptive system in such a way that the correct execution can be guaranteed. It assumes that task priorities are fixed for a given set of tasks and are not dynamically changed during execution. It assumes that there are sufficient task priority levels for the task set and that the task set models periodic events only. This means that an interrupt that is generated by a serial port peripheral is modelled as an event that occurs on a periodic rate determined by the data rate, for example. Asynchronous events such as a user pressing a key are handled differently as will be explained later.

The key policy within RMS is that tasks with shorter execution periods are given the highest priority within the system. This means that the faster executing tasks can pre-empt the slower periodic tasks so that they can meet their deadlines. The advantage this gives the system designer is that it is easier to theoretically specify the system so that the tasks will meet their deadlines without overloading the processor. This requires detailed knowledge about each task and the time it takes to execute. This and its periodicity can be used to calculate the processor loading.

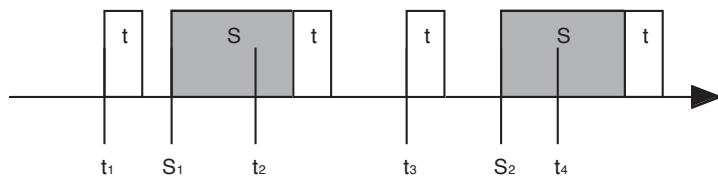
To see how this policy works, consider the examples shown in the diagram on the next page. In the diagrams, events that start a task are shown as lines that cross the horizontal time line and tasks are shown as rectangles whose length determines their execution time. Example 1 shows a single periodic task where the task t is executed with a periodicity of time t . The second example adds a second task S where its periodicity is longer than that of task t . The task priority shown is with task S having the highest priority. In this case, the RMS policy has not been followed because the longest task has been given a higher priority than the shortest task. However, please note that in this case the system works fine because of the timing of the tasks' periods.

Example 3 shows what can go wrong if the timing is changed and the periodicity for task S approaches that of task t . When t_3 occurs, task t is activated and starts to run. It does not complete because S_2 occurs and task S is swapped-in due to its

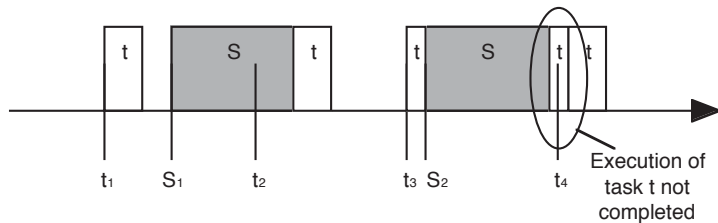
higher priority. When task S completes, task t resumes but during its execution, the event t_4 occurs and thus task t has failed to meet its task 3 deadline. This could result in missed or corrupted data, for example. When task t completes, it is then reactivated to cope with the t_4 event.



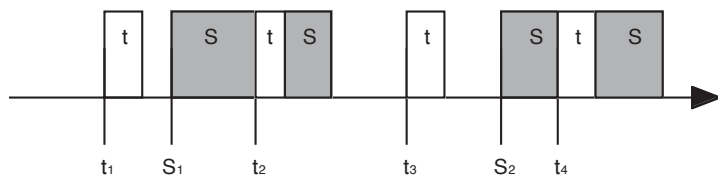
Example 1: Single periodic task



Example 2: Two periodic tasks (S and t)



Example 3: Two periodic tasks (S and t)
S has highest priority



Example 4: Two periodic tasks (S and t)
t has highest priority

Using RMS policies

Example 4 shows the same scenario with the task priorities reversed so that task t pre-empts task S. In this case, RMS policy has been followed and the system works fine with both tasks meeting their deadlines. This system is useful to understand and does allow theoretical analysis before implementation to prevent a design from having to manually assign task priorities to get it to work using a trial and error approach. It is important to remember within any calculations to take into account the context swapping

time needed to pre-empt and resume tasks. The processor utilisation can also be calculated and thus give some idea of the performance needed or how much performance is spare. Typically, the higher the utilisation (>80%), the more chance that the priority assignment is wrong and that the system will fail. If the utilisation is below this, the chances are that the system will run correctly.

This is one of the problems with theoretical analysis of such systems in that the actual design may have to break some of the assumptions on which the analysis is based. Most embedded systems will have asynchronous events and tasks running. While these can be modelled as a periodic task that polls for the asynchronous event and are analysed as if they will run, other factors such as cache memory hit ratios can invalidate the timing by lengthening the execution time of any analysis. Similarly, the act of synchronising tasks by inter-task communication can also cause difficulties within any analysis.

Deadline monotonic scheduling

Deadline monotonic scheduling (DMS) is another task priority policy that uses the nearest deadline as the criterion for assigning task priority. Given a set of tasks, the one with the nearest deadline is given the highest priority. This means that the scheduling or designer must now know when these deadlines are to take place. Tracking and, in fact, getting this information in the first place can be difficult and this is often the reason behind why deadline scheduling is often a second choice compared to RMS.

Priority guidelines

With a system that has a large number of tasks or one that has a small number of priority levels, the general rule is to assign tasks with a similar period to the same level. In most cases, this does not effect the ability to schedule correctly. If a task has a large context, i.e. it has more registers and data to be stored compared to other tasks, it is worth raising its priority to reduce the context switch overhead. This may prevent the system from scheduling properly but can be a worthwhile experiment.

Priority inversion

It is also possible to get a condition called priority inversion where a lower priority task can continue running despite there being a higher priority task active and waiting to pre-empt.

This can occur if the higher priority task is in some way blocked by the lower priority task through it having to wait until a message or semaphore is processed. This can happen for several reasons.

Disabling interrupts

While in the interrupt service routine, all other interrupts are disabled until the routine has completed. This can cause a

problem if another interrupt is received and held pending. What happens is that the higher priority interrupt is held pending in favour of the lower priority one — albeit that it occurred first. As a result, priority inversion takes place until interrupts are re-enabled at which point the higher priority interrupt will start its exception processing, thus ending the priority inversion.

One metric of an operating system is the longest period of time that all interrupts are disabled. This must then be added to any other interrupt latency calculation to determine the actual latency period.

Message queues

If a message is sent to the operating system to activate a task, many systems will process the message and then reschedule accordingly. In this way, the message queue order can now define the task priority. For example, consider an ISR that sends an unblocking message to two tasks, A and B, that are blocked waiting for the message. The ISR sends the message for task A first followed by task B. The ISR is part of the RTOS kernel and therefore may be subject to several possible conditions:

- Condition 1
Although the message calls may be completed, their action may be held pending by the RTOS so that any resulting pre-emption is stopped from switching out the ISR.
- Condition 2
The ISR may only be allowed to execute a single RTOS call and in doing so the operating system itself will clean up any stack frames. The operating system will then send messages to tasks notifying them of the interrupt and in this way simulate the interrupt signal. This is normally done through a list.

These conditions can cause priority inversion to take place. With condition 1, the ISR messages are held pending and processed. The problem arises with the methodology used by the operating system to process the pending messages. If it processes all the messages, effectively unblocking both tasks before instigating the scheduler to decide the next task to run, all is well. Task B will be scheduled ahead of task A because of its higher priority. The downside is the delay in processing all the messages before selecting the next task.

Most operating systems, however, only have a single call to process and therefore in normal operation do not expect to handle multiple messages. In this case, the messages are handled individually so that after the first message is processed, task A would be unblocked and allowed to execute. The message for task B would either be ignored or processed as part of the housekeeping at the next context switch. This is where priority inversion would occur. The ISR has according to its code unblocked both tasks and

thus would expect the higher priority task B to execute. In practice, only task A is unblocked and is running, despite it being at a lower priority. This scenario is a programming error but one that is easy to make.

To get around this issue, some RTOS implementations restrict an ISR to making either one or no system calls. With no system calls, the operating system itself will treat the ISR event as an internal message and will unblock any task that is waiting for an ISR event. With a single system call, a task would take the responsibility for controlling the message order to ensure that priority inversion does not take place.

Waiting for a resource

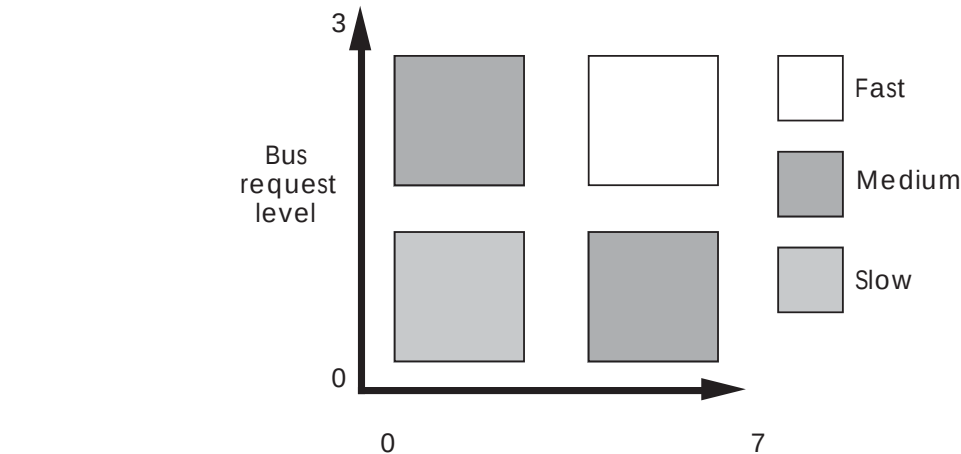
If a resource is shared with a low priority task and it does not release it, a higher priority task that needs it can be blocked until it is released. A good example of this is where the interrupt handler is distributed across a system and needs to access a common bus to handle the interrupt. This can be the case with a VMEbus system, for example.

VMEbus interrupt messages

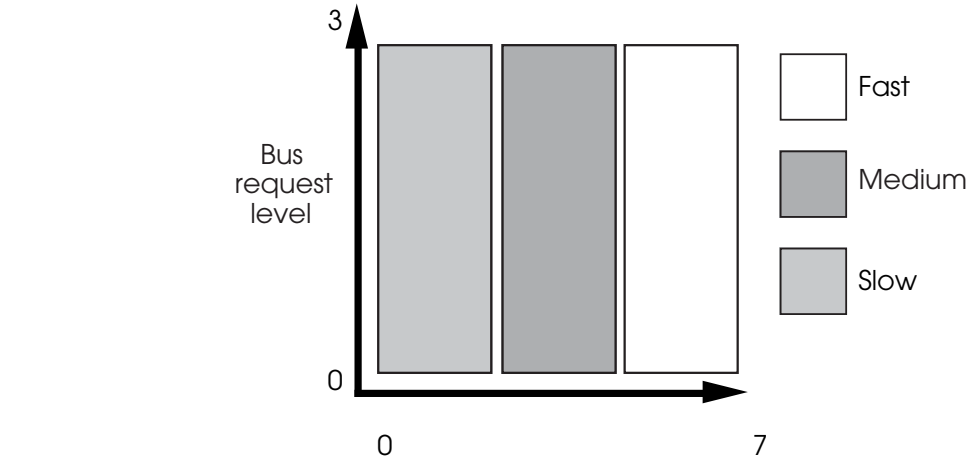
VMEbus is an interconnection bus that was developed in the early 1980s for use within industrial control and other real-time applications. The bus is asynchronous and is very similar to that of the MC68000. It comprises of a separate address, data, interrupt and control buses.

If a VMEbus MASTER wishes to inform another that a message is waiting or urgent action is required, a VMEbus interrupt can be generated. The VMEbus supports seven interrupt priority levels to allow prioritisation of a resource.

Any board can generate an interrupt by asserting one of the levels. Interrupt handling can either be centralised, and handled by one MASTER, or can be distributed among many. For multi-processor applications, distributed handling allows rapid direct communication to individual MASTERS by any board in the system capable of generating an interrupt: the MASTER that has been assigned to handle the interrupt requests the bus and starts the interrupt acknowledgement cycle. Here, careful consideration of the arbitration level chosen for the MASTER is required. The interrupt response time depends on the time taken by the handler to obtain the bus prior to the acknowledgement. This has to be correctly assigned to achieve the required performance. If it has a low priority, the overall response time may be more than that obtained for a lower priority interrupt whose handler has a higher arbitration level. The diagrams below show the relationship for both priority and round robin arbitration schemes. Again, as with the case with arbitration schemes, the round robin system has been assumed on average to provide equal access for all the priority levels.



VMEbus interrupt response times for a priority arbitration scheme



VMEbus interrupt response times for a round robin arbitration scheme

Priority inversion can occur if a lower priority interrupt may be handled in preference to a higher priority one, simply because of the arbitration levels. To obtain the best response, high priority interrupts should only be assigned to high arbitration level MASTERS. The same factors, such as local traffic on the VMEbus and access time, increase the response time as the priority level decreases.

VMEbus only allows a maximum of seven separate interrupt levels and this limits the maximum number of interrupt handlers to seven. For systems with larger numbers of MASTERS, polling needs to be used for groups of MASTERS assigned to a single interrupt level. Both centralised and distributed interrupt handling schemes have their place within a multiprocessor system. Distributed schemes allow interrupts to be used to pass high priority messages to handlers, giving a fast response time which might be critical for a real-time application. For simpler designs, where there is a dominant controlling MASTER, one handler may be sufficient.

Fairness systems

There are times when the system requires different characteristics from those originally provided or alternatively wants a system response that is not priority based. This can be achieved by using a fairness system where the bus access is distributed across the requesting processors. There are various schemes that are available such as round-robin where access is simply passed from one processor to another. Other methods can use time sharing where the bus is given to a processor on the understanding that it must be relinquished within a maximum time frame.

These type of systems can affect the interrupt response because bus access is often necessary to service an interrupt.

Tasks, threads and processes

This section of the chapter discusses the nomenclature given to the software modules running within the RTOS environment. Typically, they have been referred to as tasks but other names such as threads and processes are also used to refer to entities within the RTOS. They are sometimes used instead as an interchangeable replacement for the term task. In practice, they refer to different aspects within the system.

So far in this chapter, a task has been used to describe an entity of work within an operating system that has control over resources. When a context switch is performed, it effectively switches to another task which takes over. Strictly speaking the context switch may include additional information that is relevant to the task such as memory management information, which is beyond the simple register swapping that is performed within the processor. As a result, the term process is often used to encompass more than a simple context switch and thus includes the additional information. The problem is that this is very similar to that of a task switch or context switch that the definitions have become blurred and almost interchangeable. A task or process has several characteristics:

- It owns or controls resources, e.g. access to peripherals and so on.
- It has threads of execution. These are paths through the code contained within the task or process. Normally, there is a single thread though but this may not always be the case. Multiple threads can be supported if the task or process can maintain separate data areas for each thread. This also requires the code to be written in a re-entrant manner.
- It requires additional information beyond the normal register contents to maintain its integrity, e.g. memory management information, cache flushing and so on. When a new process or task is swapped-in, not only are the processor registers changed but additional work must be done

such as invalidating caches to ensure that the new process or task does not access incorrect information.

A thread has different characteristics:

- It has no additional context information beyond that stored in the processor register set.
- Its ownership of resources is inherited from its parent task or process.

With a simple operating system, there is no difference between the thread context switch and the process level switch. As a result, these terms almost become interchangeable. With a multi-user, multitasking operating system, this is not the case. The process or task is the higher level with the thread(s) the lower level.

Some operating systems take this a stage further and define a three level hierarchy: a process consists of a set of tasks with each task having multiple threads. Be warned! These terms mean many different things depending on who is using them.

Exceptions

With most embedded systems, access to the low level exception handler is essential to allow custom routines to be written to support the system. This can include interrupt routines to control external peripherals, emulation routines to simulate instructions or facilities that the processor does not support — software floating point is a very good example of this — and other exception types.

Some of these exceptions are needed by the RTOS to provide entry points into the kernel and to allow the timers and other facilities to function. As a result, most RTOSs already provide the basic functionality for servicing exceptions and provide access points into this functionality to allow the designer to add custom exception routines. This can be done in several ways:

- Patching the vector table

This is relatively straight forward if the vector is not used by the RTOS. If it is, then patching will still work but the inserted user exception routine must preserve the exception context and then jump to the existing handler instead of using a return from exception type instruction to restore normal processing. If it is sharing an exception with the RTOS, there must be some form of checking so that the user handler does not prevent the RTOS routine from working correctly.

- Adding user routines to existing exception handlers

This is very similar to the previous technique in that the user routine is added to any existing RTOS routine. The difference is that the mechanism is more formal and does not require vector table patching or any particular checking by the user exception handler.

- Generating a pseudo exception that is handled by separate user exception handler(s)

This is even more formal — and slower — and effectively replaces the processor level exception routine with a RTOS level version in which the user creates his own vector table and exception routines. Typically, all this is performed through special kernel calls which register a task as the handler for a particular exception. On completion, the handler uses a special return from the exception call into the RTOS kernel to signify that it has completed.

Memory model

The memory model that the processor offers can and often varies with the model defined by the operating system and is open to the software designer to use. In other words, although the processor may support a full 32 bit address range with full memory mapped I/O and read/write access anywhere in the map at a level of an individual word or map, the operating system's representation of the memory map may only be 28 bits, with I/O access allocated on a 512 byte basis with read only access for the first 4 Mbytes of RAM and so on.

This discrepancy can get even wider, the further down in the levels that you go. For example, most processors that have sophisticated cache memory support use the memory management unit. This then requires fairly detailed information about the individual memory blocks within a system. This information has to be provided to the RTOS and is normally done using various memory allocation techniques where information is provided when the system software is compiled and during operation.

Memory allocation

Most real-time operating systems for processors where the memory map can be configured, e.g. those that have large memory addressing and use memory mapped I/O, get around this problem by using a special file that defines the memory map that the system is expected to use and support. This will define which memory addresses correspond to I/O areas, RAM, ROM and so on. When a task is created, it will be given a certain amount of memory to hold its code area and provide some initial data storage. If it requires more memory, it will request it from the RTOS using a special call such as `malloc()`. The RTOS will look at the memory request and allocate memory by passing back a pointer to the additional memory. The memory request will normally define the memory characteristics such as read/write access, size, and even its location and attributes such as physical or logical addressing.

The main question that arises is why dynamically allocate memory? Surely this can be done when the tasks are built and

included with the operating system? The answer is not straightforward. In simple terms, it is a yes and for many simple embedded systems, this is correct. For more complex systems, however, this static allocation of memory is not efficient, in that memory may be reserved or allocated to a task yet could only be used rarely within the system's operation. By dynamically allocating memory, the total amount of memory that the system requires for a given function can be reduced by reclaiming and reallocating memory as and when needed by the system software. This will be explained in more detail later on.

Memory characteristics

The memory characteristics are important to understand especially when different memory addresses correspond to different physical memory. As a result, asking for a specific block of memory may impact the system performance. For example, consider an embedded processor that has both internal and external memory. The internal memory is faster than the external memory and therefore improves performance by not inserting wait states during a memory access. If a task asks for memory expecting to be allocated internal memory but instead receives a pointer to external memory, the task performance will be degraded and potentially the system can fail. This is not a programming error in the true sense of the word because the request code and RTOS have executed correctly. If the request was not specific enough, then the receiving task should expect the worst case type of memory. If it does not or needs specific memory, this should be specified during the request. This is usually done by specifying a memory address or type that the RTOS memory allocation code can check against the memory map file that was used when the system was built.

- Read/write access
This is straightforward and defines the access permissions that a task needs to access a memory block.
- Internal/external memory
This is normally concerned with speed and performance issues. The different types of memory are normally defined not by their speed but indirectly through the address location. As a result, the programmer must define and use a memory map so that the addresses of the required memory block match up the required physical memory and thus its speed. Some RTOSs actually provide simple support flags such as internal/external but this is not common.
- Size
The minimum and maximum sizes are system dependent and typically are influenced by the page size of any memory management hardware that may be present. Some systems can return with partial blocks, e.g. if the original request was for 8 kbytes, the RTOS may only have 4 kbytes free and

instead of returning an error, will return a pointer to the 4 kbytes block instead. This assumes that the requesting task will check the returned size and not simply assume that because there was no error, it has all the 8 kbytes it requested! Check the RTOS details carefully.

- I/O

This has several implications when using processors that execute instructions out of order to remove pipeline stalls and thus gain performance. Executing instructions that access I/O ports out of sequence can break the program syntax and integrity. The program might output a byte and then read a status register. If this is reversed, the correct sequence has been destroyed and the software will probably crash. By declaring I/O addresses as I/O, the processor can be programmed to ensure the correct sequence whenever these addresses are accessed.

- Cached or non-cachable

This is similar to the previous paragraph on I/O. I/O addresses should not be cached to prevent data corruption. Shared memory blocks need careful handling with caches and in many cases unless there is some form of bus snooping to check that the contents of a cache is still valid, these areas should also not be cached.

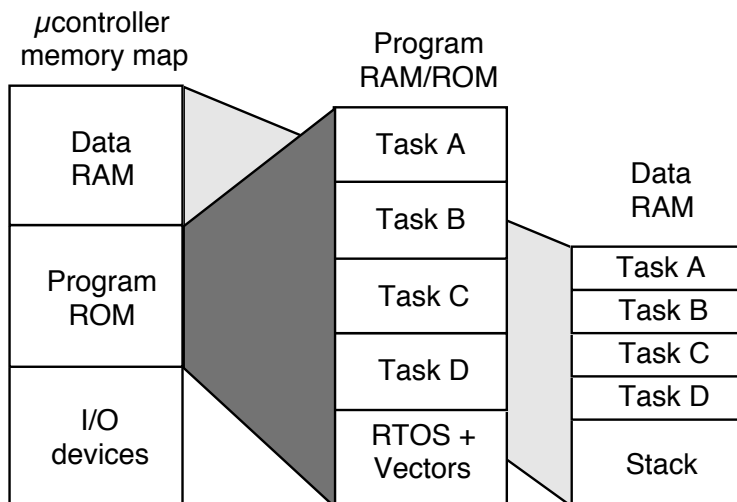
- Coherency policies

Data caches can have differing coherency policies such as write-through, copy back and so on which are used to ensure the data coherency within the system. Again, the ability to specify or change these policies is useful.

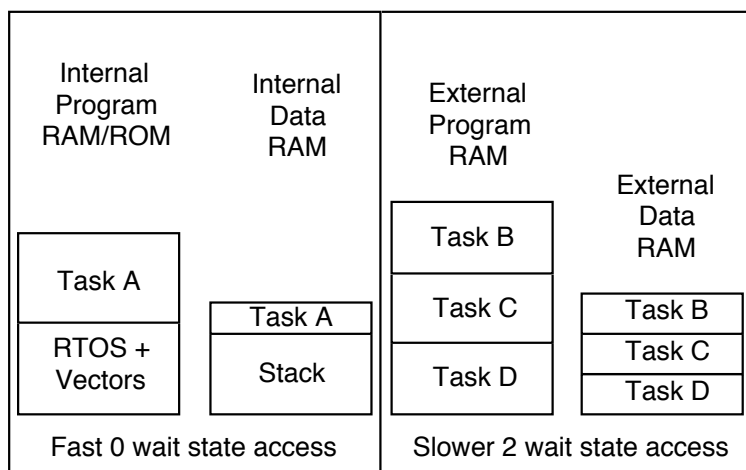
Example memory maps

The first example is that commonly used within a simple microcontroller where its address space is split into the different memory types. The example shows three: I/O devices and peripherals, program RAM and ROM and data RAM. The last two types have then been expanded to show how they could be allocated to a simple embedded system. The program area contains the code for four tasks, the RTOS code and the processor vector table. The data RAM is split into five areas: one for each of the tasks and a fifth area for the stack. In practice, these areas are often further divided into internal and external memory, EPROM and EEPROM, SRAM and even DRAM, depending on the processor architecture and model. This example uses a fixed static memory map where the memory requirements for the whole system are defined at compile and build time. This means that tasks cannot get access to additional memory by using some of the memory allocated to another task. In addition, it should be remembered that although the memory map shows nicely partitioned areas, it does not imply nor should it be assumed that task A cannot access task C's data

area, for example. In these simple processors and memory maps, all tasks have the ability to access any memory location and it is only the correct design and programming that ensures that there is no corruption. Hardware can be used to provide this level of protection but it requires some form of memory management unit to check that programs are conforming to their design and not accessing memory that they should not. Memory management is explained in some detail in the next section.



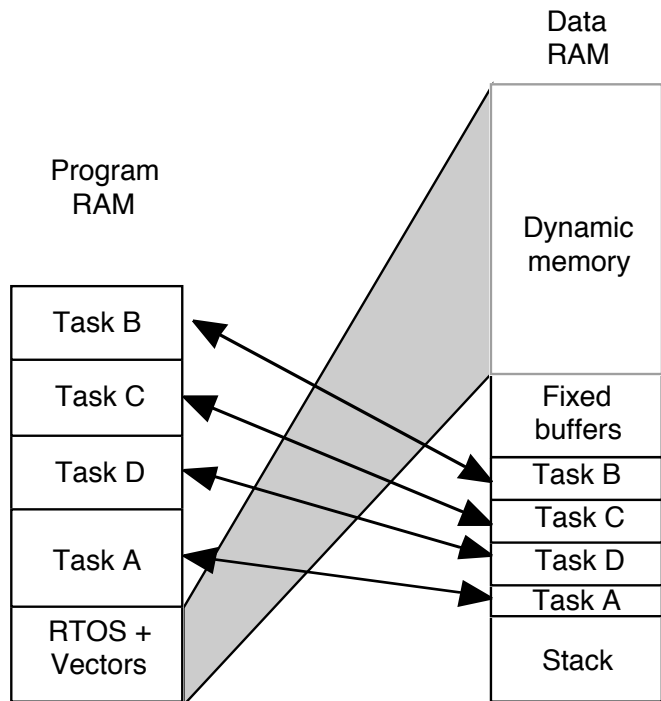
Simple microcontroller memory map



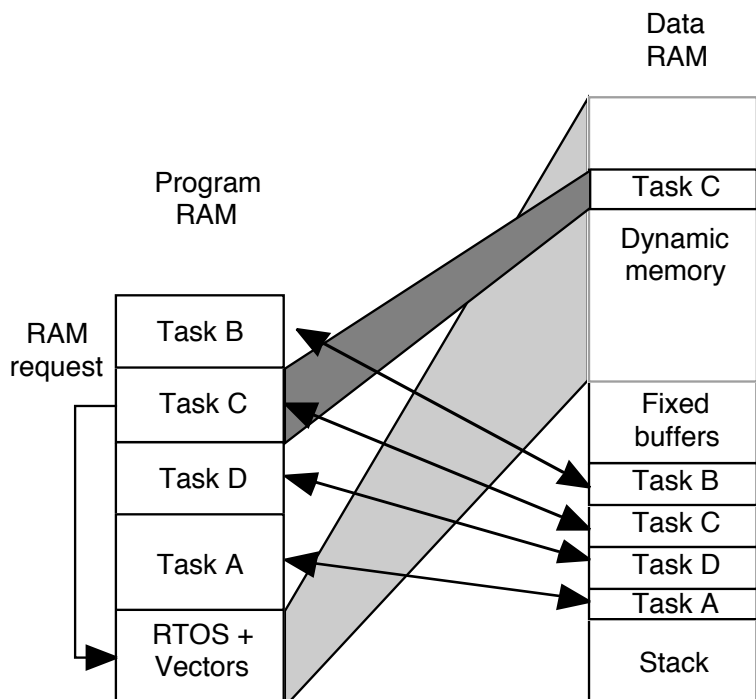
Internal and external memory map

The second example shows a similar system to the first example except that it has been further partitioned into internal and external memory. The internal memory runs faster than the external memory and because it does not have any wait states, its access time is faster and the processor performance does not degrade. The slower external memory has two wait states and with a single cycle processor would degrade performance by 66%

— each instruction would take three clocks instead of one, for example.



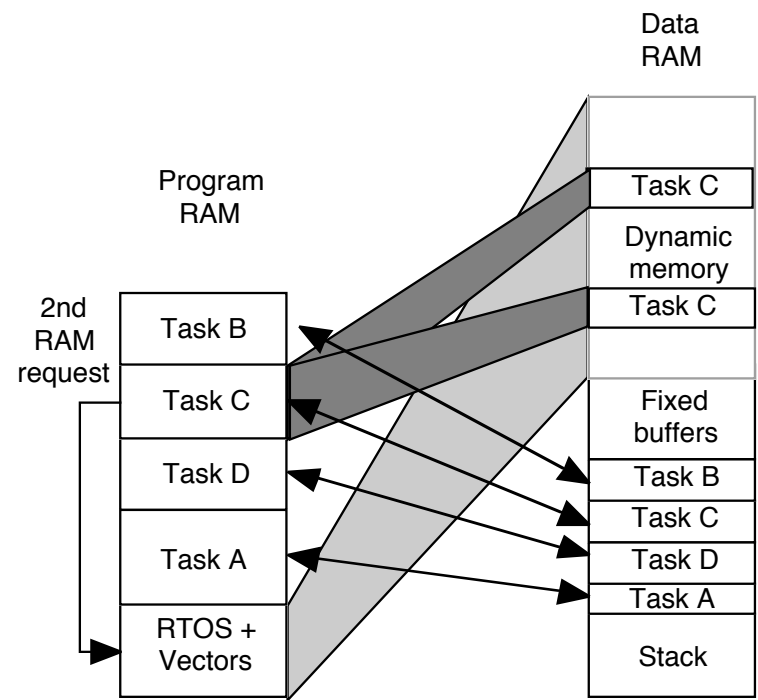
Memory map with dynamic allocation — initial state



Memory map with dynamic allocation — after memory request

Given this performance difference, it is important that the memory resources are carefully allocated. In the example, task A requires the best performance and the system needs fast task switching. This means that both the task A code and data, along with the RTOS and the system stack, are allocated to the internal memory where it will get the best performance. All other task code and data are stored externally because all the internal memory is used.

The third example shows a dynamic allocation system where tasks can request additional memory as and when they need it. The first map shows the initial state with the basic memory allocated to tasks and RTOS. This is similar to the previous examples except that there is a large amount of memory entitled to dynamic memory which is controlled by the RTOS and can be allocated dynamically by it to other tasks on demand. The next two diagrams show this in operation. The first request by task C starts by sending a request to the RTOS for more memory. The RTOS allocates a block to the task and returns a pointer to it. Task C can use this to get access to this memory. This can be repeated if needed and the next diagram shows task C repeating the request and getting access to a second block. Blocks can also be relinquished and returned to the RTOS for allocation to other tasks at some other date. This process is highly dynamic and thus can provide a mechanism for minimising memory usage. Task C could be allocated all the memory at certain times and as the memory is used and no longer required, blocks can be reallocated to different tasks.



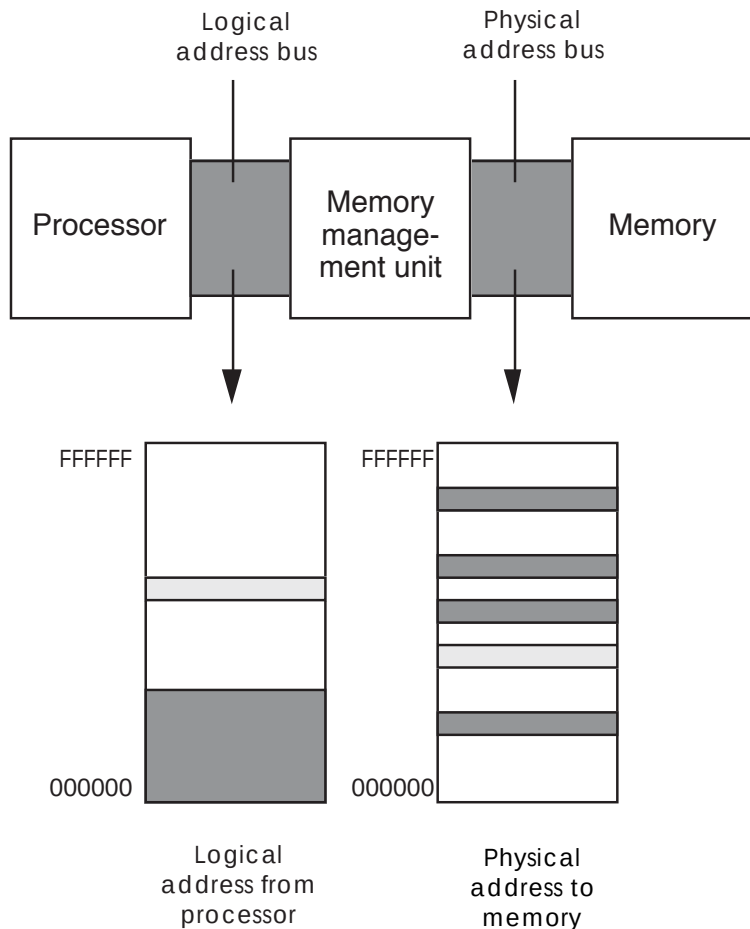
Memory map with dynamic allocation — after 2nd memory request

The problem with this is in calculating the minimum amount of memory that the system will require. This can be difficult to estimate and many designs start with a large amount of memory, get the system running and then find out empirically the minimum amount of required memory.

In this section, the use of memory management within an embedded design has been alluded to in the case of protecting memory for corruption. While this is an important use, it is a secondary advantage compared to its ability to reuse memory through address translation. Before returning to the idea of memory protection, let's consider how address translation works and affects the memory map.

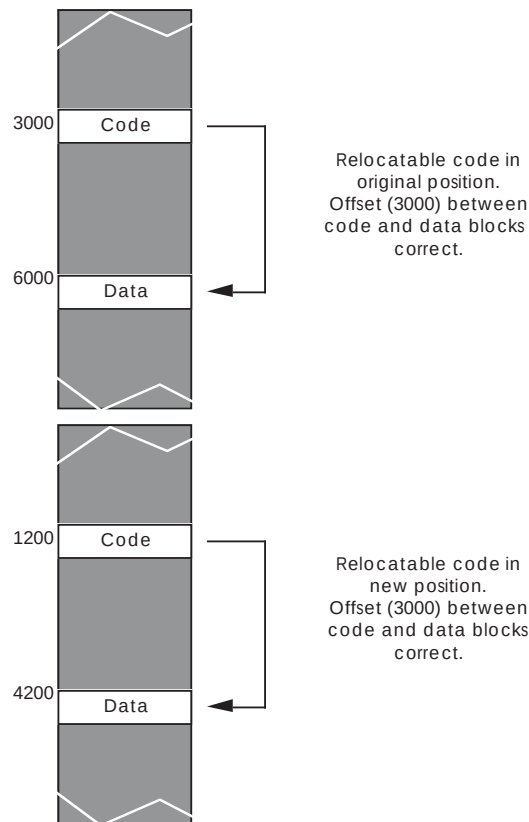
Memory management address translation

While the use of memory management usually implies the use of an operating system to remove the time-consuming job of defining and writing the driver software, it does not mean that every operating system supports memory management.



Many do not or are extremely limited in the type of memory management facilities that they support. For operating systems that do support it, the designer can access standard software that controls the translation of logical addresses to different physical addresses as shown in the diagram.

In this example, the processor thinks that it is accessing memory at the bottom of its memory map, while in reality it is being fetched from different locations in the main memory map. The memory does not even need to be contiguous: the processor's single block or memory can be split into smaller blocks, each with a different translation address.

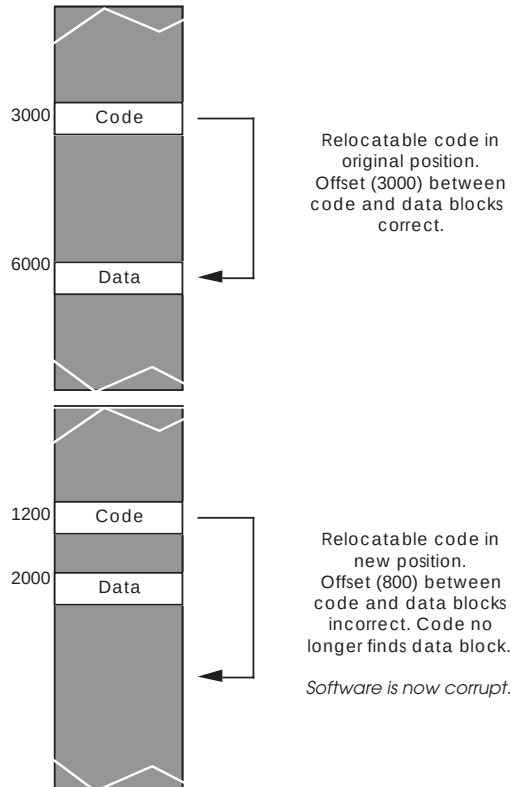


Correct movement of relocatable code

This address translation is extremely powerful and allows the embedded system designer many options to provide greater fault detection and/or security within the system or even cost reduction through the use of virtual memory. The key point is that memory management divides the processor memory map into definable regions with different properties such as read and write only access for one way data transfers and task or process specific memory access.

If no memory management hardware is present, most operating systems can replace their basic address translation

facility with a software-based scheme, provided code is written to be position independent and relocatable. The more sophisticated techniques start to impose a large software overhead which in many cases is hard to justify within a simple system. Address translation is often necessary to execute programs in different locations from that in which they were generated. This allows the reuse of existing software modules and facilitates the easy transfer of software from a prototype to a final system.



Incorrect movement of relocatable code

The relocation techniques are based on additional software built into the program loader or even into the operating system itself. If the operating system program loader cannot allocate the original memory, the program is relocated into the next available block and the program allowed to execute. Relocatable code does not have any immediate addressing values and makes extensive use of program relative addressing. Data areas or software subroutines are not referenced explicitly but are located by relative addressing modes using offsets:

- Explicit addressing
e.g. branch to subroutine at address \$0F04FF.
- Relative addressing
e.g. branch to subroutine which is offset from here by \$50 bytes.

Provided the offsets are maintained, then the relative addressing will locate data and code wherever the blocks are located in memory. Most modern compilers will use these techniques but do not assume that all of them do.

There are alternative ways of manipulating addresses to provide address translation, however.

Bank switching

When the first 8 bit processors became available with their 64 kbytes memory space, there were many comments concerning whether there was a need for this amount of memory. When the IBM PC appeared with its 640 kbyte memory map, the same comments were made and here we are today using PCs that probably have 32 Mbytes of RAM. The problem faced by many of the early processor architectures with a small 64 kbyte memory map is how the space can be expanded without having to change the architecture and increase register sizes and so on.

One technique that is used is that of bank switching. With this technique, additional bits are used to select different banks of memory. Each bank is the full 64 kbytes in size and is used in the normal way. By writing to the individual selection bits, an individual bank can be selected and used. It could be argued that this is no different from adding additional address bits. Using two selection bits will support four 64 kbyte banks giving a total memory space of 256 kbytes. This is the same amount of memory that can be addressed by increasing the number of address bits from 16 to 18. The difference, however, is that adding address bits implies that the programming model and processor knows about the wider address space and can use it. With bank switching, this is not the case, and the control and manipulation of the banks is under the control of the program that the processor is running. In other words, the processor itself has no knowledge that bank switching is taking place. It simply sees the normal 64 kbyte address space.

This approach is frequently used with microcontrollers that have either small external address spaces or alternatively limited external address buses. The selection bits are created by dedicating bits from the microcontroller's parallel I/O lines and using these to select and switch the external memory banks. The bank switching is controlled by writing to these bits within the I/O port.

This has some interesting repercussions for designs that use a RTOS. The main problem is that the program must understand when the system context is safe enough to allow a bank switch to be made. This means that system entities such as data structures, buffers and anything else that is stored in memory including program software must fit into the boundaries created by the bank switching.

This can be fairly simple but it can also be extremely complex. If the bank switching is used to extend a database, for example, the switching can be easy to control by inserting a check

for a memory bank boundary. Records 1–100 could be in bank A, with bank B holding records 101–200. By checking the record number, the software can switch between the banks as needed. Such an implementation could define a subroutine as the access point to the data and within this routine the bank switching is managed so that it is transparent to the rest of the software.

Using bank switching to support large stacks or data structures on the other hand is more difficult because the mechanisms that use the data involve both automatic and software controlled access. Interrupts can cause stacks to be incremented automatically and there is no easy way of checking for an overflow and then incorporating the bank switching and so on needed to use it.

In summary, bank switching is used and there are 8 bit processors that have dedicated bits to support it but the software issues to use it are immense. As a result, it is frequently left for the system designer to figure out a way to use it within a design. As a result, few, if any, RTOS environments support this memory model.

Segmentation

Segmentation can be described as a form of bank switching that the processor architecture does know about! It works by providing a large external address bus but maintaining the smaller address registers and pointers within the original 8 bit architecture. To bridge the gap, the memory is segmented internally into smaller blocks that match the internal addressing and additional registers known as segment registers are used to hold the additional address data needed to complete the larger external address.

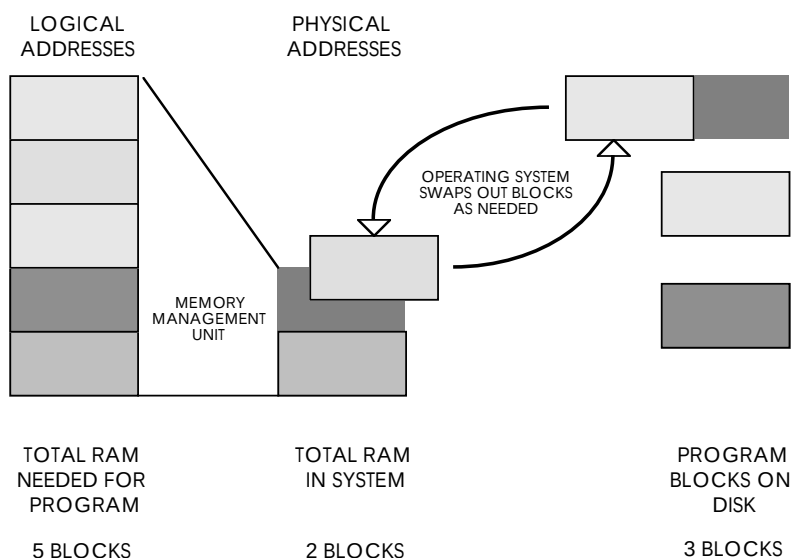
Probably the most well-known implementation is the Intel 8086 architecture.

Virtual memory

With the large linear addressing offered by today's 32 bit microprocessors, it is relatively easy to create large software applications which consume vast quantities of memory. While it may be feasible to install 64 Mbytes of RAM in a workstation, the costs are expensive compared with 64 Mbytes of a hard disk. As the memory needs go up, this differential increases. A solution is to use the disk storage as the main storage medium, divide the stored program into small blocks and keep only the blocks in processor system memory that are needed. This technique is known as virtual memory and relies on the presence within the system of a memory management unit.

As the program executes, the MMU can track how the program uses the blocks, and swap them to and from the disk as needed. If a block is not present in memory, this causes a page fault and forces some exception processing which performs the swapping operation. In this way, the system appears to have large

amounts of system RAM when, in reality, it does not. This virtual memory technique is frequently used in workstations and in the UNIX operating system. Its appeal in embedded systems is limited because of the potential delay in accessing memory that can occur if a block is swapped out to disk.



Using virtual memory to support large applications

Choosing an operating system

Comparing an operating system from 10 years ago with one offered today shows how operating system technology has developed over the past years. Although the basic functions provided by the old and the newer operating systems — they all provide multitasking, real-time responses and so on — are still present, there have been some fundamental changes in the improvement in the ease of use, performance and debugging facilities. Comparing a present-day car with one from the 1920s is a good analogy. The basic mechanics and principles have largely remained unchanged — that is, the engine, gearbox, brakes, transmission — but there has been a great improvement in the ease of driving, comfort and facilities. This is similar to what has happened with operating systems. The basic mechanisms of context switches, task control blocks, linked lists and so on are the basic fundamentals of any operating system or kernel.

As a result, it can be quite difficult to select an operating system. To make such a choice, it is necessary to understand the different ways that operating systems have developed over the years and the advantages that this has brought. The rest of this chapter discusses these changes and can help formulate the criteria that can be used to make a decision.

Assembler versus high level language

In the early 1980s, operating systems were developed in response to minicomputer operating systems where the emphasis was on providing the facilities and performance offered by minicomputers. To achieve performance, they were often written in assembler rather than in a high level language such as C or PASCAL. The reason for this was simply one of performance: compiler technology was not advanced enough to provide the compact and fast code needed to run an operating system. For example, many compilers from the early 1980s did not use all the M68000 address and data registers and limited themselves to only one or two. The result was code that was extremely inefficient when compared with hand coded assembler which did use all the processor's registers.

The disadvantage is that assembler code is harder to write and maintain compared to a high level language and is extremely difficult to migrate to other architectures. In addition, the interface between the operating system and a high level language was not well developed and in some cases non-existent! Writing interface libraries was considered part of the software task.

As both processor performance and compiler technology improved, it became feasible to provide an operating system written in a high level language such as C which provided a seamless integration of the operating system interface and application development language.

The choice of using assembler or a high level language with some assembler compared to using an integrated operating system and high level language is fairly obvious. What was acceptable a few years ago is no longer the case and today's successful operating systems are highly integrated with their compiler technology.

ROMable code

With early operating systems, restrictions in the code development often prevented operating systems and compilers from generating code that could be blown into read only memory for an embedded application. The reasons were often historic rather than technical, although the argument that most applications were too big to fit into the relatively small size of EPROM that was available was certainly true for many years. Today, most users declare this requirement as mandatory, and it is a standard offering from compilers and operating system vendors alike.

Scheduling algorithms

One area of constant debate is that of the scheduling algorithms that are used to select which task is to execute next. There are several different approaches which can be used. The first is to switch tasks only at the end of a time slice. This allows a fairer distribution of the processing power across a large number of

tasks but at the expense of response time. Another is to take the first approach but allow certain events to force switch a task even if the current one has not used up all its allotted time slice. This allows external interrupts to get a faster response. Another event that can be used to interrupt the task is an operating system call.

Others have implemented priority systems where a task's priority and status within the ready list can be changed by itself, the operating system or even by other tasks. Others have a fixed priority system where the level is fixed when the task is created. Some operating systems even allow different scheduling algorithms to be implemented so that a designer can change them to give a specific response.

Changing algorithms and so on are usually indicative of trying to squeeze the last bit of performance from the system and in such cases it may be better to use a faster processor, or even in extreme cases actually accept that the operating system cannot meet the required performance and use another.

Pre-emptive scheduling

One consistent requirement that has appeared is that of pre-emptive scheduling. This refers to a particular scheduling algorithm where the highest priority task will interrupt or pre-empt a currently executing task irrespective of whether it has used its allotted time slice, and will continue running until a higher level task is ready to. This gives the best response to interrupts and events but can be a little dangerous. If a task is given the highest priority and does not lower its priority or pre-empt itself, then other tasks will not get an opportunity to execute. Therefore the ability to pre-empt is often restricted to special tasks with time critical routines.

Modular approach

The idea of reusing code whenever possible is not a new one but it can be difficult to implement. Obvious candidates with an operating system are device drivers for I/O , and kernels for different processors. The key is in defining a standard interface which allows such modules to be reused without having to alter or change the code. This means that memory maps must not be hardwired, or assumptions made by the driver or operating system. One of the problems with early versions of many operating systems was the fact that it was not until fairly late in their development that a modular approach for device drivers was available. As a result, the standard release included several drivers for the same peripheral chip, depending on which VMEbus board it was located.

Today, this approach is no longer acceptable and operating systems are more modular in their approach and design. The advantages for users are far more compact code, shorter development times and the ability to reuse code. A special driver can be reused without modification. This coupled with the need to keep up

with the number of boards that need standard ports has led to the development of automated build systems that can take modular drivers and create a new version extremely quickly.

Re-entrant code

This follows on from the previous topic but has one fundamental difference in that a re-entrant software module can be shared by many tasks and also interrupted at any point and reused without any problems. For example, consider module A which is shared by two tasks B and C. If task B uses module A and exits from it, the module will be left in a known state, ready for another task to use it. If task C starts to use it and in the middle of its execution is switched out in favour of task B, then the problem may appear. If task B starts to use module A, it may experience problems because A will be in an indeterminate state. Even if it does not, other problems may still be lurking. If module A uses global variables, then task B will cause them to be reset. When task C returns to continue execution, its global data will have been destroyed.

A re-entrant module can tolerate such interruptions without experiencing these types of problems. The golden rule is for the module to only access data associated with the task that is using the module code. Variables are stored on stacks, in registers or in memory areas specific to the task and not to the module. If shared modules are not re-entrant, care must be taken to ensure that a context switch does not occur during its execution. This may mean disabling or locking out the scheduler or dispatcher.

Cross-development platforms

Today, most software development is done on cross-development platforms such as Sun workstations, UNIX systems and IBM PCs. This is in direct contrast to early systems which required a dedicated software development system. The degree of platform support and the availability of good development tools which go beyond the standard of symbolic level debug have become a major product selling point.

Integrated networking

This is another area which is becoming extremely important. The ability to use a network such as TCP/IP on Ethernet to control target boards, download code and obtain debugging information is fast becoming a mandatory requirement. It is rapidly replacing the more traditional method of using serial RS232 links to achieve the same end.

Multiprocessor support

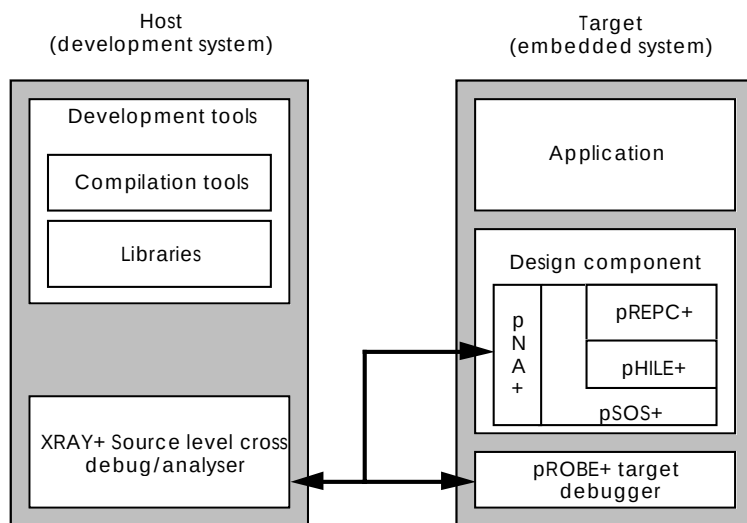
This is another area which has changed dramatically. Ten years ago it was possible to use multiple processors provided the developer designed and coded all the inter-processor communication. Now, many of today's operating systems can provide

optional modules that will do this automatically. However, multiprocessing techniques are often misunderstood and as this is such a big topic for both hardware and software developers it is treated in more depth later in this text.

Commercial operating systems

pSOS⁺

pSOS⁺ is the name of a popular multitasking real-time operating system. Although the name refers to the kernel itself, it is often used in a more generic way to refer to a series of development tools and system components. The best way of looking at the products is to use the overall structure as shown in the diagram. The box on the left is concerned with the development environment while that on the right are the software components that are used in the final target system. The two halves work together via communication links such as serial lines, Ethernet and TCP/IP protocol or even over the VMEbus itself.



pSOS⁺ overall structure

pSOS⁺ kernel

The kernel supports a wide range of processor families like the Motorola M68000 family, the Intel 80x86 range, and the M88000 and i960 RISC processors. It is small in size and typically takes about 15–20 kbytes of RAM, although the final figure will depend on the configuration and processor type.

It supports more than 50000 system objects such as tasks, memory partitions, message queues and so on and will execute time-critical routines consistently irrespective of the application size. In other words, the time to service a message queue is the

same irrespective of the size of the message. Note that this will refer to the time taken to pass the message or perform the service only and does not and cannot take into account the time taken by the user to handle messages. In other words, the consistent timing refers to the message delivery and not the actions taken as a result of the message. Worst case figures for interrupt latency and context switch for an MC68020 running at 25 MHz are 6 and 19 μ s respectively. Among its 55 service calls, it provides support for:

- Task management
- Message queues
- Event services
- Semaphore services
- Asynchronous services
- Storage allocation services
- Time management and timer services
- I/O supervisor services
- Interrupt management
- Error handling services

pSOS multiprocessor kernel

pSOS⁺^m is the multiprocessing version of the kernel. From an application or task's perspective, it is virtually the same as the single processor version except that the kernel now has the ability to send and receive system objects from other processors within the system. The application or task does not know where other tasks actually reside or the communication method used to link them.

The communication mechanism works in this way. Each processor runs its own copy of the kernel and has a kernel interface to the media used for linking the processors, either a VMEbus backplane or a network. When a task sends a message to a local task, the local processor will handle it. If the message is for task running on another node, the local operating system will look up the recipient's identity and location. The message is then routed across the kernel interface across to the other processor and its operating system where the message is finally delivered. Different kernel interfaces are needed for different media.

pREPC⁺ runtime support

This is a compiler independent run-time environment for C applications. It is compatible with the ANSI X3J11 technical committee's proposal for C run-time functionality and provides 88 functions that can be called from C programs. Services supported include formatted I/O, file I/O and string manipulation.

pREPC⁺ is not a standalone product because it uses pSOS⁺ or pSOS⁺^m for device I/O and task functions and calls pHILE⁺ for file and disk I/O. Its routines are re-entrant which allows multiple tasks to use the same routine simultaneously.

pHILE⁺ file system

This product provides file I/O and will support either the MS-DOS file structure or its own proprietary formats. The MS-DOS structure is useful for data interchange while the proprietary format is designed to support the highest data throughput across a wide range of devices. pHILE⁺ does not drive physical devices directly but provides logical data via pSOS⁺ to a device driver task that converts this information to physical data and drives the storage device.

pNA⁺ network manager

This is a networking option that provides TCP/IP communication over a variety of media such as Ethernet and FDDI. It conforms to the UNIX 4.3 BSD socket syntax and approach and is compatible with other TCP/IP-based networking standards such as ftp and NFS.

As a result, pNA⁺ is used to provide efficient downloading and debugging communication between the target and a host development system. Alternatively, it can be used to provide a communication path between other systems that are also sitting on the same network.

pROBE⁺ system level debugger

This is the system level debugger which provides the system and low level debugging facilities. With this, system objects can be inspected or even used to act as breakpoints if needed. It can use either a serial port to communicate with the outside world or if pNA⁺ is installed, use an TCP/IP link instead.

XRAY⁺ source level debugger

This is a complementary product to pROBE⁺ as it can use the debugger information and combine it with the C source and other symbolic information on the host to provide a complete integrated debugging environment.

OS-9

OS-9 was originally developed by Microware and Motorola as a real-time operating system for the Motorola MC6809 8 bit processor and it appeared on many 6809-based systems such as the Exorset 165 and the Dragon computer. It provided a true hierarchical filing system and the ability to run multiple tasks. It has since been ported to the M68000 family and the Intel 80x86 processor families. It is best described as a complete operating system with its own user commands, interface and so on. Unlike other products which have concentrated on the central kernel and then built outwards but stopping at below the user and utility level, OS-9 goes from a multi-user multitasking interface with a range of utilities down to the low level kernel. Early on it sup-

ported UNIX by using and supporting the same library interface and similar system calls. So much so that one of its strengths was the ability to take UNIX source code, recompile it and then run it.

One criticism has been its poor real-time response although a new version has been released which used a smaller, compact and faster kernel to provide better performance. The full facilities are still provided by the addition of other kernel services around the inner one. It provides more sophisticated support such as multimedia extensions which other operating systems do not, and because of this and its higher level of utilities and expansion has achieved success in the marketplace.

VXWorks

VXWorks has taken another approach to the problem of providing a real-time environment as well as standard software tools and development support. Instead of creating its own or reproducing a UNIX-like environment, it actually has integrated with UNIX to provide its development and operational environment. Through VXWorks' UNIX compatible networking facilities, it can combine with UNIX to form a complete run-time solution as well. The UNIX system is responsible for software development and non-real-time debugging while the VXWorks kernel is used for testing, debugging and executing the real-time applications, either standalone or part of a network with UNIX.

How does this work? The UNIX system is used as the development host and is used to edit, compile, link and administer the building of real-time code. These modules can then be burned into ROM or loaded from disk and executed standalone under the VXWorks kernel. This is possible because VXWorks can understand the UNIX object module format and has a UNIX compatible software interface. By using the standard UNIX pipe and socket mechanisms to exchange data between tasks and by using UNIX signals for asynchronous events, there is no need for recompilation or any other conversion routines. Instead, the programmer can use the same interface for both UNIX and VXWorks software without having to learn different libraries or programming commands. It supports the POSIX 1003.4 real-time extensions and multiprocessing support for up to 20 processors is offered via another option called VxMP.

The real key to VXWorks is its ability to network with UNIX to allow a hybrid system to be developed or even allow individual modules or groups to be transferred to run in a VXWorks environment. The network can be over an Ethernet or even using shared memory over a VMEbus, for example.

VRTX-32

VRTX-32 from Microtec Research has gained a reputation for being an extremely compact but high performance real-time kernel. Despite being compact — typically about 8 kbytes of code

for an MC68020 system — it provides task management facilities, inter-task communication, memory control and allocation, clock and timer functions, basic character I/O and interrupt handling facilities.

Under the name of VRTXvelocity, VRTX-32 systems can be designed using cross-development platforms such as Sun workstations and IBM PCs. The systems can be integrated with the host, usually using an Ethernet, to provide an integrated approach to system design.

IFX

Its associated product IFX (input/output file executive) provides support for more complicated I/O subsystems such as disks, terminals, and serial communications using structures such as pipes, null devices, circular buffers and caches. The file system is MS-DOS compatible although if this is not required, disks can be treated as single partitions to speed up response.

TNX

This is the TCP/IP networking package that allows nodes to communicate with hosts and other applications over the Ethernet. The Ethernet device itself can either be resident on the processor board or accessible across a VMEbus. It supports both stream and datagram sockets.

RTL

This is the run-time library support for Microtec and Sun compilers and provides the library interface to allow C programs to call standard I/O functions and make VRTX-32 calls.

RTscope

This is the real-time multitasking debugger and system monitor that is used to debug VRTX tasks and applications. It operates on two levels: the board level debugger provides the standard features such as memory and register display and modify, software upload and download and so on. In the VRTX-32 system monitor mode, tasks can be interrogated, stopped, suspended and restarted.

MPV

The multiprocessor VRTX-32 extensions allow multiple processors each running their own copy of VRTX to pass messages and other task information from one processor to another and thus create a multiprocessor system. The messages are based across the VMEbus using shared memory although other links such as RS232 or Ethernet are possible.

LynxOS-POSIX conformance

POSIX (IEEE standard portable operating system interface for computer environments) began in 1986 as an attempt to provide an open standard for operating system support. The ideas

behind it are to provide vendor independence, protection from technical obsolescence, the availability of standard off-the-shelf applications, the preservation of software investment and to provide connectivity between computers.

It is based on UNIX but has added a set of real-time extensions as defined in the POSIX 1003.4 document. These cover a more sophisticated semaphore system which uses the `open()` call to create them. This call is more normally associated with opening a file. The facilities include persistent semaphores which retain their binary state after their last use, and the ability to force a task to wait for a semaphore through a locking mechanism.

The extensions also provide a process or task locking mechanism which prevents memory pages associated with the task or process from being swapped out to memory, thus improving the real-time response for critical routines. Shared memory is better supported through the provision of the `shmmap()` call which will allocate a sheared memory block. Both asynchronous and synchronous I/O and inter-task message passing are supported along with real-time file extensions to speed up file I/O. This uses techniques such as preallocating file space before it is required.

At the time of writing LynxOS is the main real-time product that supports these standards, although many others support parts of the POSIX standard. Indeed, there is an increasing level of support for this type of standardisation.

However, it is not a complete panacea and, while any attempt for standardisation should be applauded, it does not solve all the issues. Real-time operating systems and applications are very different in their needs, design and approach, as can be seen from the diversity of products that are available today. Can all of these be met by a single standard? In addition, the main cost of developing software is not in porting but in testing and documenting the product and this again is not addressed by the POSIX standards. POSIX conformance means that software should be portable across processors and platforms, but it does not guarantee it. With many of today's operating systems available in versions for the major processor families, is the POSIX portability any better? Many of these questions are yet to be answered conclusively by supporters or protagonists.

An alternative way of looking at this problem is: do you assume that a ported real-time product will work because it is POSIX compliant without testing it on the new target? In most cases the answer will be no and that testing will be needed. What POSIX conformance has given is a helping hand in the right direction and this should not be belittled, neither should it be seen as a miracle cure. In the end, the success of the POSIX standards will depend on the market and users seeing benefit in its approach. It is an approach that is gathering pace, and one that the real-time market should be aware of. It is possible that it may succeed where other attempts at a real-time interface standard have failed. Another possibility for POSIX conformance is Windows NT.

Windows NT

Windows NT has been portrayed as many different things during its short lifetime. When it first appeared, it was perceived by many as the replacement for Windows 3.1, an alternative to UNIX, and finally has settled down as an operating system for workstations, servers and power users. This chameleon-like change was not due to any real changes in the product but were caused by a mixture of aspirations and misunderstandings.

Windows NT is rapidly replacing Windows 3.1 and Windows 95 and parts of its technology have already found themselves incorporated into Windows 95 and Windows for Workgroups. Whether the replacement is through a merging of the operating system technologies or through a sharing of common technology, only time will tell. The important message is that the Windows NT environment is becoming prevalent, especially with Microsoft's aim of a single set of programming interfaces that will allow an application to run on any of its operating system environments. Its greater stability and reliability is another feature that is behind its adoption by many business systems in preference over Windows 95. All this is fine, but how does this fit with an embedded system?

There are several reasons why Windows NT is being used in real-time environments. It may not have the speed of a dedicated RTOS but it has the important features and coupled with a fast processor, reasonable performance.

- Portability

Most PC-based operating systems were written in low-level assembler language instead of a high level language such as C or C++. This decision was taken to provide smaller programs sizes and the best possible performance. The disadvantage is that the operating system and applications are now dependent on the hardware platform and it is extremely difficult to move from one platform to another. MS-DOS is written in 8086 assembler which is incompatible with the M68000 processors used in the Apple Macintosh. For a software company like Microsoft, this has an additional threat of being dependent on a single processor platform. If the platform changes — who remembers the Z80 and 6502 processors which were the mainstays of the early PCs — then its software technology becomes obsolete. With an operating system that is written in a high level language and is portable to other platforms, it allows Microsoft and other application developers to be less hardware dependent.

- True multitasking

While more performant operating systems such as UNIX and VMS offer the ability to run multiple applications

simultaneously, this facility is not really available from the Windows and MS-DOS environments (a full explanation of what they can do and the difference will be offered later in this chapter). This is now becoming a very important aspect for both users and developers alike so that the full performance of today's processors can be utilised.

- Multi-threaded

Multi-threading refers to a way of creating software that can be reused without having to have multiple copies of the code or memory spaces. This leads to more efficient use of both memory and code.

- Processor independent

Unlike Windows and MS-DOS which are completely linked to the Intel 80x86 architecture, Windows NT through its portability is processor independent and has been ported to other processor architectures such as Motorola's PowerPC, DEC's Alpha architecture and MIPS RISC processor systems.

- Multiprocessor support

Windows NT uses a special interface to the processor hardware which makes it independent of the processor architecture that it is running on. As a result, this not only gives processor independence but also allows the operating system to run on multiprocessor systems.

- Security and POSIX support

Windows NT offers several levels of security through its use of a multi-part access token. This token is created and verified when a user logs onto the system and contains IDs for the user, the group he is assigned to, privileges and other information. In addition, an audit trail is also provided to allow an administrator to check who has used the system, when they used it and what they did. While an overkill for a single user, this is invaluable with a system that is either used by many or connected to a network.

The POSIX standard defines a set of interfaces that allow POSIX compliant applications to easily be ported between POSIX compliant computer systems.

Both security and POSIX support are commercially essential to satisfy purchasing requirements from government departments, both in the US and the rest of the world.

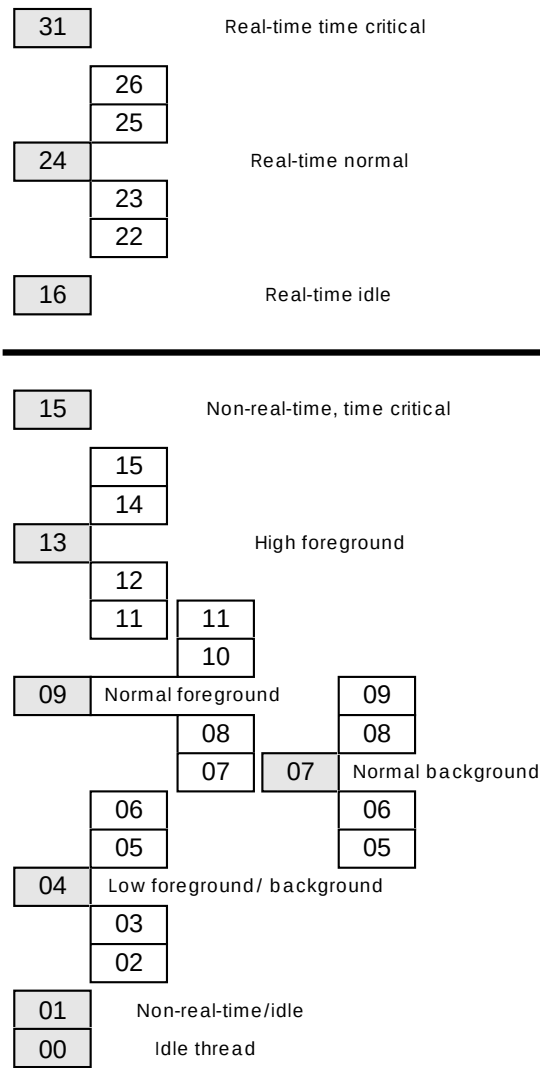
Windows NT characteristics

Windows NT is a pre-emptive multitasking environment that will run multiple applications simultaneously and uses a priority based mechanism to determine the running order. It is capable of providing real-time support in that it has a priority mechanism and fast response times for interrupts and so on, but

it is less deterministic — there is a wider range of response times — when compared to a real-time operating system such as pSOS or OS-9 used in industrial applications. It can be suitable for many real-time applications with less critical timing characteristics and this is a big advantage over the Windows 3.1 and Windows 95 environments. It is interesting to note that this technology now forms the backbone of all the Windows software environments.

Process priorities

Windows NT calls all applications, device drivers, software tasks and so on processes and this nomenclature will be used from now on. Each process can be assigned one of 32 priority levels which determines its scheduling priority. The 32 levels are divided into two groups called the real-time and dynamic classes.



Windows NT priority levels — base class levels are shaded

The real-time classes comprise priority levels 16 through to 31 and the dynamic classes use priority levels 15 to 0. Within these two groups, certain priorities are defined as base classes and processes are allocated a base process. Independent parts of a process — these are called threads — can be assigned their own priority levels which are derived from the base class priority and can be ± 2 levels different. In addition, a process cannot move from a real-time class to a dynamic one.

The diagram shows how the base classes are organised. The first point is that within a given dynamic base class, it is possible for a lot of overlap. Although a process may have a lower base class compared to another process, it may be at a higher priority than the other one depending on the actual priority level that has been assigned to it. The real-time class is a little simpler although again there is some possibility for overlap.

User applications like word processors, spread sheets and so on run in the dynamic class and their priority will change depending on the application status. Bring an application from the background to the foreground by expanding the shrunk icon or by switching to that application will change its priority appropriately so that it gets allocated a higher priority and therefore more processing. Real-time processes include device drivers handling the keyboard, cursor, file system and other similar activities.

Interrupt priorities

The concept of priorities is not solely restricted to the pre-emption priority previously described. Those priorities come into play when an event or series of events occur. The events themselves are also controlled by 32 priority levels defined by the hardware abstraction layer (HAL).

Interrupt	Description
31	Hardware error interrupt
30	Powerfail interrupt
29	Inter-processor interrupt
28	Clock interrupt
27-12	Standard IBM PC AT interrupt levels 0 to 15
11-4	Reserved (not generally used)
3	Software debugger
2-0	Software interrupts for device drivers etc.

Interrupt priorities

The interrupt priorities work in a similar way to those found on a microprocessor: if an interrupt of a higher priority than the current interrupt priority mask is generated, the current processing will stop and be replaced by the associated routines for the new higher priority level. In addition, the mask will be raised to match that of the higher priority. When the higher priority processing has been completed, the previous processing will be restored and allowed to continue. The interrupt priority mask will also be restored to its previous value.

Within Windows NT, the interrupt processing is also subject to the multitasking priority levels. Depending on how these are assigned to the interrupt priority levels, the processing of a high priority interrupt may be delayed until a higher priority process has completed. It makes sense therefore to have high priority interrupts processed by processes with high priority scheduling levels. Comparing the interrupts and the priority levels shows that this maxim has been followed. Software interrupts used to communicate between processes are allocated both low interrupt and scheduling priorities. Time critical interrupts such as the clock and inter-processor interrupts are handled as real-time processes and are allocated the higher real-time scheduling priorities.

The combination of both priority schemes provides a fairly complex and flexible method of structuring how external and internal events and messages are handled.

Resource protection

If a system is going to run multiple applications simultaneously then it must be able to ensure that one application doesn't affect another. This is done through several layers of resource protection. Resource protection within MS-DOS and Windows 3.1 is a rather hit and miss affair. There is nothing to stop an application from directly accessing an I/O port or other physical device and if it did so, it could potentially interfere with another application that was already using it. Although the Windows 3.1 environment can provide some resource protection, it is of collaboration level and not mandatory. It is without doubt a case of self-regulation as opposed to obeying the rules of the system.

Protecting memory

The most important resource to protect is memory. Each process is allocated its own memory which is protected from interference by other processes through programming the memory management unit. This part of the processor's hardware tracks the executing process and ensures that any access to memory that it has not been allocated or given permission to use is stopped.

Protecting hardware

Hardware such as I/O devices are also protected by the memory management unit and any direct access is prevented. Such accesses have to be made through a device driver and in this way the device driver can control who has access to a serial port and so on. A mechanism called a spinlock is also used to control access. A process can only access a device or port if the associated spinlock is not set. If someone else is using it, the process must wait until they have finished.

Coping with crashes

If a process crashes then it is important for the operating system to maintain as much of the system as possible. This requires that the operating system as well as other applications must have its own memory and resources given to it. To ensure this is the case, processes that are specific to user applications are run in a user mode while operating system processes are executed in a special kernel mode. These modes are kept separate from each other and are protected. In addition, the operating system has to have detailed knowledge of the resources used by the crashed process so that it can clean up the process, remove it and thus free up the resources that it used. In some special cases, such as power failures where the operating system may have a limited amount of time to shut down the system in a controlled manner or save as much of the system data as it can, resources are dedicated specifically for this functionality. For example, the second highest interrupt priority is allocated to signalling a power failure.

Windows NT is very resilient to system crashes and while processes can crash, the system will continue. This is essentially due to the use of user and kernel modes coupled with extensive resource protection. Compared to Windows 3.1 and MS-DOS, this resilience is a big advantage.

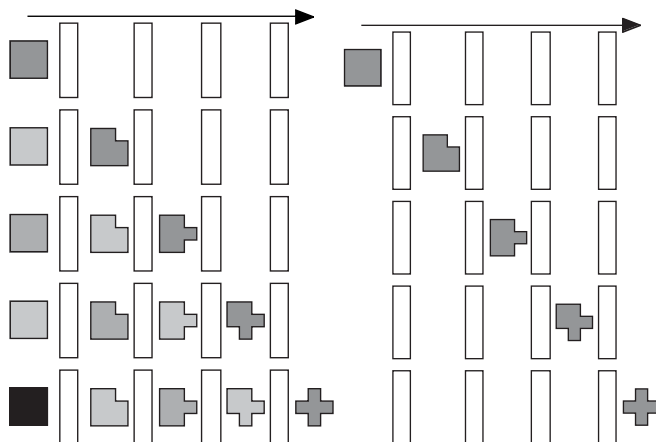
Multi-threaded software

There is a third difference with Windows NT that many other operating systems do not provide in that it supports multi-threaded processes. Processes can support several independent processing paths or threads. A process may consist of several independent sections and thus form several different threads in that the context of the processing in one thread may be different from that in another thread. In other words, the process has all the resources defined that it will use and if the process can support multi-threaded operations, the scheduler will see multiple threads going through the process. A good analogy is a production line. If the production line is single threaded, it can only produce a single end product at a time. If it is multi-threaded, it separates the production process into several independent parts and each part can work on a different product. As soon as the first operation has taken place, a second thread can be started. The threads do not have to follow the same path and can vary their route through the process.

The diagram shows a simple multi-threaded operation with each thread being depicted by a different shading. As the first thread progresses through, a second thread can be started. As that progresses through, a third can commence and so on. The resources required to process the multiple threads in this case are the same as if only one thread was supported.

The advantage of multi-threaded operation is that the process does not have to be duplicated every time it is used: a new

thread can be started. The disadvantage is that the process programming must ensure that there is no contention or conflict between the various threads that it supports. All the threads that exist in the process can access each other's data structures and even files. The operating system which normally polices the environment is powerless in this case. Threads within Windows NT derive their priority from that of the process although the level can be adjusted within a limited range.



Multi-threaded (left) and single-threaded (right) operations

Addressing space

The addressing space within Windows NT is radically different from that experienced within MS-DOS and Windows 3.1. It provides a 4 Gbyte virtual address space for each process which is linearly addressed using 32 bit address values. This is different from the segmented memory map that MS-DOS and Windows have to use. A segmented memory scheme uses 16 bit addresses to provide address spaces of only 64 kbytes. If addresses beyond this space have to be used, special support is needed to change the segment address to point to a new segment. Gone are the different types of memory such as extended and expanded.

This change towards a large 32 bit linear address space improves the environment for software applications and increases their performance and capabilities to handle large data structures. The operating system library that the applications use is called WIN32 to differentiate it from the WIN16 libraries that Windows 3.1 applications use. Applications that use the WIN32 library are known as 32 bit or even native — this term is also used for Windows NT applications that use the same instruction set as the host processor and therefore do not need to emulate a different architecture.

To provide support for legacy MS-DOS and Windows 3.1 applications, Windows NT has a 16 bit environment which simulates the segmented architecture that these applications use.

Virtual memory

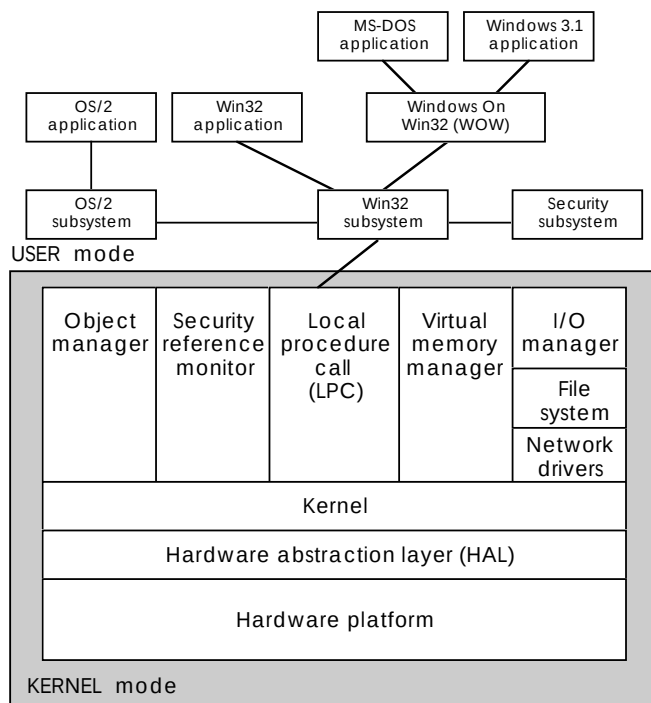
The idea behind virtual memory is to provide more memory than physically present within the system. To make up the shortfall, a file or files are used to provide overflow storage for applications which are too big to fit in the system RAM at one time. Such applications' memory requirements are divided into pages and unused pages are stored on disk.

When the processor wishes to access a page which is not resident in memory, the memory management hardware asserts a page fault, selects the least used page in memory and swaps it with the wanted page stored on disk. Therefore, to reduce the system overhead, fast mass storage and large amounts of RAM are normally required.

Windows NT uses a swap file to provide a virtual memory environment. The file is dynamic in size and varies with the amount of memory that all the software including the operating system, device driver, and applications require. The Windows 3.1 swap file is limited to about 30 Mbytes in size and this effectively limits the amount of virtual memory that it can support.

The internal architecture

The internal architecture is shown in the diagram below and depicts the components that run in the user and kernel modes. Most of the operating system runs in the kernel mode with the exception of the security and WIN32 subsystems.



The internal Windows NT architecture

The environments are protected in that direct hardware access is not allowed and that a single application will run in a single environment. This means that some applications or combinations may not work in the Windows NT environment. On the other hand, running them in separate isolated environments does prevent them from causing problems with other applications or the operating system software.

Virtual memory manager

The virtual memory manager controls and supervises the memory requirements of an operating system. It allocates to each process a private linear address space of 4 Gbytes which is unique and cannot be accessed by other processes. This is one reason why legacy software such as Windows 3.1 applications run as if they are the only application running.

With each process running in its own address space, the virtual memory manager ensures that the data and code that the process needs to run is located in pages of physical memory and ensures that the correct address translation is performed so that process addresses refer to the physical addresses where the information resides. If the physical memory is all used up, the virtual memory manager will move pages of data and code out to disk and store it in a temporary file. With some physical memory freed up, it can load from disk previously stored information and make it available for a process to use. This operation is closely associated with the scheduling process which is handled within the kernel. For an efficient operating system, it is essential to minimise the swapping out to disk — each disk swap impacts performance — and the most efficient methods involve a close correlation with process priority. Low priority processes are primary targets for moving information out to disk while high priority processes are often locked into memory so that they offer the highest performance and are never swapped out to disk. Processes can make requests to the virtual memory manager to lock pages in memory if needed.

User and kernel modes

Two modes are used to isolate the kernel and other components of the operating system from any user applications and processes that may be running. This separation dramatically improves the resilience of the operating system. Each mode is given its own addressing space and access to hardware is made through the operating system kernel mode. To allow a user process access, a device driver must be used to isolate and control its access to ensure that no conflict is caused.

The kernel mode processes use the 16 higher real-time class priority levels and thus operating system processes will take preference over user applications.

Local procedure call (LPC)

This is responsible for co-ordinating system calls from an application and the WIN32 subsystem. Depending on the type of call and to some extent its memory needs, it is possible for applications to be routed directly to the local procedure call (LPC) without going through the WIN32 subsystem.

The kernel

The kernel is responsible for ensuring the correct operation of all the processes that are running within the system. It provides the synchronisation and scheduling that the system needs. Synchronisation support takes the form of allowing threads to wait until a specific resource is available such as an object, semaphore, an expired counter or other similar entity. While the thread is waiting it is effectively dormant and other threads and processes can be scheduled to execute.

The scheduling procedures use the 32 level priority scheme previously described in this chapter and is used to schedule threads rather than processes. With a process potentially supporting multiple threads, the scheduling operates on a thread basis and not on a process basis as this gives a finer granularity and control. Not scheduling a multi-threaded process would affect several threads which may not be the required outcome. Scheduling on a thread basis gives far more control.

Interrupts and other similar events also pass through the kernel so that it can pre-empt the current thread and reschedule a higher priority thread to process the interrupt.

File system

Windows NT supports three types of file system and these different file systems can co-exist with each other although there can be some restrictions if they are accessed by non-Windows NT systems across a network, for example.

- **FAT**
File allocation table is the file system used by MS-DOS and Windows 3.1 and uses file names with an 8 character name and a 3 character extension. The VFAT system used by Windows 95 and supports long file names is also supported with Windows NT v4 in that it can read Windows 95 long file names.
- **HPFS**
High performance file system is an alternative file system used by OS/2 and supports file names with 254 characters with virtually none of the character restrictions that the FAT system imposes. It also uses a write caching to disk technique which stores data temporarily in RAM and writes it to disk at a later stage. This frees up an application from

waiting until the physical disk write has completed. The physical disk write is performed when the processor is either not heavily loaded or when the cache is full.

- NTFS

The NT filing system is Windows NT's own filing system which conforms to various security recommendation and allows system administrators to restrict access to files and directories within the filing system.

All three filing systems are supported — Windows NT will even truncate and restore file names that are not MS-DOS compatible — and are selected during installation.

Network support

As previously stated, Windows NT supports most major networking protocols and through its multi-tasking capabilities can support several simultaneously using one or more network connections. The drivers that do the actual work are part of the kernel and work closely with the file system and security modules.

I/O support

I/O drivers are also part of the kernel and these provide the link between the user processes and threads and the hardware. MS-DOS and Windows 3.1 drivers are not compatible with Windows NT drivers and one major difference between Windows NT and Windows 3.1 is that not all hardware devices are supported. Typically modern devices and controllers can be used but it is wise to check the existence of a driver before buying a piece of hardware or moving from Windows 3.1 to Windows NT.

HAL approach

The hardware abstraction layer (HAL) is designed to provide portability across different processor-based platforms and between single and multi-processor systems. In essence, it defines a piece of virtual hardware that the kernel uses when it needs to access hardware or processor resources. The HAL layer then takes the virtual processor commands and requests and translates them to the actual processor system that it is actually using. This may mean a simple mapping where a Windows NT interrupt level corresponds to a processor hardware interrupt but it can involve a complete emulation of a different processor. Such is the case to support MS-DOS and Windows 3.1 applications where an Intel 80x86 processor is emulated so that an Intel instruction set can be run.

With the rest of Windows NT being written in C, a portable high level language, the only additional work to the recompilation and testing is to write a suitable HAL layer for the processor platform that is being used.

Linux

Linux started as a personal interest project by Linus Torvalds at the University of Helsinki in Finland to produce an operating system that looked and felt like UNIX. It was based on work that he had done in porting Minix, an operating system that had been shipped with a textbook that described its inner workings.

After much discussion via user groups on the Internet, the first version of Linux saw the light of day on the 5 October, 1991. While limited in its abilities — it could run the GNU bash shell and gcc compiler but not much else — it prompted a lot of interest. Inspired by Linus Torvalds' efforts, a band of enthusiasts started to create the range of software that Linux offers today. While this was progressing, the kernel development continued until some 18 months later, when it reached version 1.0. Since then it has been developed further with many ports for different processors and platforms. Because of the large amount of software available for it, it has become a very popular operating system and one that is often thought of as a candidate for embedded systems.

However it is based on the interfaces and design of the UNIX operating system which for various reasons is not considered suitable for embedded design. If this is the case, how is it that Linux is now forging ahead in the embedded world. To answer this question, it is important to understand how it came about and was developed. That means starting with the inspiration behind Linux, the UNIX operating system.

Origins and beginnings

UNIX was first described in an article published by Ken Thompson and Dennis Ritchie of Bell Research Labs in 1974, but its origins owe much to work carried out by a consortium formed in the late 1960s, by Bell Telephones, General Electric and the Massachusetts Institute of Technology, to develop MULTICS — a MULTIplexed Information and Computing Service. Their goal was to move away from the then traditional method of users submitting work in as punched cards to be run in batches — and receiving their results several hours (or days!) later. Each piece of work (or job) would be run sequentially — and this combination of lack of response and the punched card medium led to many frustrations — as anyone who has used such machines can confirm. A single mistake during the laborious task of producing punched cards could stop the job from running and the only help available to identify the problem was often a 'syntax error' message. Imagine how long it could take to debug a simple program if it took the computer several hours to generate each such message!

The idea behind MULTICS was to generate software which would allow a large number of users simultaneous access to the computer. These users would also be able to work interactively

and on-line in a way similar to that experienced by a personal computer user today. This was a fairly revolutionary concept. Computers were very expensive and fragile machines that required specially trained staff to keep other users away from and protect *their* machine. However, the project was not as successful as had been hoped and Bell dropped out in 1969. The experience gained in the project was turned to other uses when Thompson and Ritchie designed a computer filing system on the only machine available — a Digital Equipment PDP-7 mini computer.

While this was happening, work continued on the GE645 computer used in the MULTICS project. To improve performance and save costs (processing time was very expensive), they wrote a very simple operating system for the PDP-7 to enable it to run a space travel game. This operating system, which was essentially the first version of UNIX, included a new filing system and a few utilities.

The PDP-7 processor was better than nothing — but the new software really cried out for a better, faster machine. The problem faced by Thompson and Ritchie was one still faced by many today. It centred on how to persuade management to part with the cash to buy a new computer, such as the newer Digital Equipment Company's PDP-11. Their technique was to interest the Bell legal department in the UNIX system for text processing and use this to justify the expenditure. The ploy was successful and UNIX development moved along.

The next development was that of the C programming language, which started out as an attempt to develop a FORTRAN language compiler. Initially, a programming language called B which was developed, which was then modified into C. The development of C was crucial to the rapid movement of UNIX from a niche within a research environment to the outside world.

UNIX was rewritten in C in 1972 — a major departure for an operating system. To maximise the performance of the computers then available, operating systems were usually written in a low level assembly language that directly controlled the processor. This had several effects. It meant that each computer had its own operating system, which was unique, and this made application programs hardware dependent. Although the applications may have been written in a high level language (such as FORTRAN or BASIC) which could run on many different machines, differences in the hardware and operating systems would frequently prevent these applications from being moved between systems. As a result, many man hours were spent porting software from one computer to another and work around this computer equivalent of the Tower of Babel.

By rewriting UNIX in C, the painstaking work of porting system software to other computers was greatly reduced and it became feasible to contemplate a common operating system running on many different computers. The benefit of this to users was

a common interface and way of working, and to software developers, an easy way to move applications from one machine to another. In retrospect, this decision was extremely far sighted.

The success of the legal text processing system, coupled with a concern within Bell about being tied to a number of computer vendors with incompatible software and hardware, resulted in the idea of using the in-house UNIX system as a standard environment. The biggest advantage of this was that only one set of applications needed to be written for use on many different computers. As UNIX was now written in a high level language, it was a lot more feasible to port it to different hardware platforms. Instead of rewriting every application for each computer, only the UNIX operating system would need to be written for each machine — a lot less work. This combination of factors was too good an opportunity to miss. In September 1973, a UNIX Development Support group was formed for the first UNIX applications, which updated telephone directory information and intercepted calls to changed numbers.

The next piece of serendipity in UNIX development was the result of a piece of legislation passed in 1956. This prevented AT&T, who had taken over Bell Telephone, from selling computer products. However, the papers that Thompson and Ritchie had published on UNIX had created a quite a demand for it in academic circles. UNIX was distributed to universities and research institutions at virtually no cost on an 'as is' basis — with no support. This was not a problem and, if anything, provided a motivating challenge. By 1977, over 500 sites were running UNIX.

By making UNIX available to the academic world in this way, AT&T had inadvertently discovered a superb way of marketing the product. As low cost computers became available through the advent of the mini computer (and, later, the microprocessor), academics quickly ported UNIX and moved the rapidly expanding applications from one machine to another. Often, an engineer's first experience of computing was on UNIX systems with applications only available on UNIX. This experience then transferred into industry when the engineer completed training. AT&T had thus developed a very large sales force promoting its products — without having to pay them! A situation that many marketing and sales groups in other companies would have given their right arms for. Fortunately for AT&T, it had started to licence and protect its intellectual property rights without restricting the flow into the academic world. Again, this was either far sighted or simply common sense, because they had to wait until 1984 and more legislation changes before entering the computer market and starting to reap the financial rewards from UNIX.

The disadvantage of this low key promotion was the appearance of a large number of enhanced variants of UNIX which had improved appeal — at the expense of some compatibility. The issue of compatibility at this point was less of an issue than today.

UNIX was provided with no support and its devotees had to be able to support it and its applications from day one. This self sufficiency meant that it was relatively easy to overcome the slight variations between UNIX implementations. After all, most of the application software was written and maintained by the users who thus had total control over its destiny. This is not the case for commercial software, where hard economic factors make the decision for or against porting an application between systems.

With the advent of microprocessors like the Motorola MC68000 family, the Intel 8086 and the Zilog Z8000, and the ability to produce mini computer performance and facilities with low cost silicon, UNIX found itself a low cost hardware platform. During the late 1970s and early 1980s, many UNIX systems appeared using one of three UNIX variants.

XENIX was a UNIX clone produced by Microsoft in 1979 and ported to all three of the above processors. It faded into the background with the advent of MS-DOS, albeit temporarily. Several of the AT&T variants were combined into System III, which, with the addition of several features, was later to become System V. The third variant came from work carried out at Berkeley (University of California), which produced the BSD versions destined to become a standard for the Digital Equipment Company's VAX computers and throughout the academic world.

Of the three versions, AT&T were the first to announce that they would maintain upward compatibility and start the lengthy process of defining standards for the development of future versions. This development has culminated in AT&T System V release 4, which has effectively brought the System V, XENIX and BSD UNIX environments together.

What distinguishes UNIX from other operating systems is its wealth of application software and its determination to keep the user away from the physical system resources. There are many compilers, editors, text processors, compiler construction aids and communication packages supplied with the basic release. In addition, packages from complete CAD and system modelling to integrated business office suites are available.

The problem with UNIX was that it was quite an expensive operating system to buy. The hardware in many cases was specific to a manufacturer and this restricted the use of UNIX. What was needed was an alternative source of UNIX. With the advent of Linux, this is exactly what happened.

Inside Linux

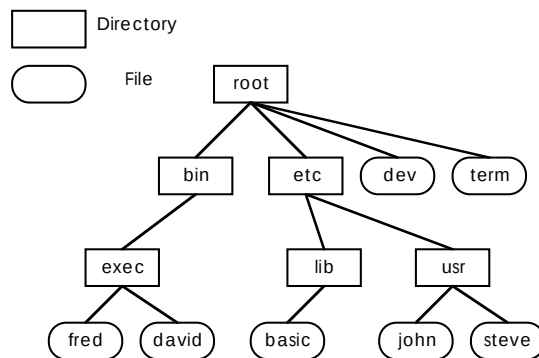
The key to understanding Linux as an operating system is to understand UNIX and then to grasp how much the operating system protects the user from the hardware it is running on. It is very difficult to know exactly where the memory is in the system, what a disk drive is called and other such information. Many facets of the Linux environment are logical in nature, in that they

can be seen and used by the user — but their actual location, structure and functionality is hidden. If a user wants to run a 20 Mbyte program on a system, UNIX will use its virtual memory capability to make the machine behave logically like one with enough memory — even though the system may only have 4 Mbytes of RAM installed. The user can access data files without knowing if they are stored on a floppy or a hard disk — or even on another machine many miles away and connected via a network. UNIX uses its facilities to present a logical picture to the user while hiding the more physical aspects from view.

The Linux file system

Linux like UNIX has a hierarchical filing system which contains all the data files, programs, commands and special files that allow access to the physical computer system. The files are usually grouped into directories and subdirectories. The file system starts with a root directory and divides it into subdirectories. At each level, there can be subdirectories that continue the file system into further levels and files that contain data. A directory can contain both directories and files. If no directories are present, the file system will stop at that level for that path.

A file name describes its location in the hierarchy by the path taken to locate it, starting at the top and working down. This type of structure is frequently referred to as a tree structure which, if turned upside down, resembles a tree by starting at a single root directory — the trunk — and branching out.



The Linux file system

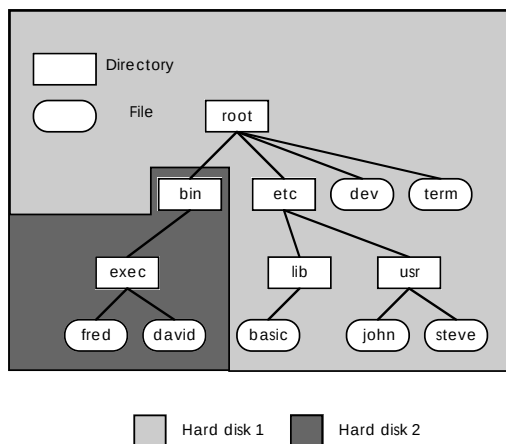
The full name, or path name, for the file *steve* located at the bottom of the tree would be */etc/usr/steve*. The */* character at the beginning is the symbol used for the starting point and is known as root or the root directory. Subsequent use within the path name indicates that the preceding file name is a directory and that the path has followed down that route. The */* character is in the opposite direction to that used in MS-DOS systems: a tongue in cheek way to remember which slash to use is that MS-DOS is backward compared with Linux — and thus its slash character points backward.

The system revolves around its file structure and all physical resources are also accessed as files. Even commands exist as files. The organisation is similar to that used within MS-DOS — but the original idea came from UNIX, and not the other way around. One important difference is that with MS-DOS, the top of the structure is always referred to by the name of the hard disk or storage medium. Accessing an MS-DOS root directory C:\ immediately tells you that drive C holds the data. Similarly, A:\ and B:\ normally refer to floppy disks. With UNIX, such direct references to hardware do not exist. A directory is simply present and rarely gives any clues as to its physical location or nature. It may be a floppy disk, a hard disk or a disk on another system that is connected via a network.

All Linux files are typically one of four types, although it can be extremely difficult to know which type they are without referring to the system documentation. A *regular* file can contain any kind of data and is not restricted in size. A *special* file represents a physical I/O device, such as a terminal. *Directories* are files that hold lists of files rather than actual data and *named pipes* are similar to regular files but restricted in size.

The physical file system

The physical file system consists of mass storage devices, such as floppy and hard disks, which are allocated to parts of the logical file system. The logical file system (previously described) can be implemented on a system with two hard disks by allocating the *bin* directory and the filing subsystem below it to hard disk no. 2 — while the rest of the file system is allocated to hard disk no. 1. To store data on hard disk 2, files are created somewhere in the *bin* directory. This is the logical way of accessing mass storage. However, all physical input and output can be accessed by sending data to special files which are normally located in the */dev* directory. This organisation of files is shown.



The file system and physical storage

This can create a great deal of confusion: one method of sending data to a hard disk is by allocating it to part of the logical file system and simply creating data files. The second method involves sending the data directly to the special */dev* file that represents the physical disk drive — which itself exists in the logical file system!

This conflict can be explained by an analogy using bookcases. A library contains many bookcases where many books are stored. The library represents the logical file system and the bookcases the physical mass storage. Books represent the data files. Data can be stored within the file system by putting books into the bookcases. Books can be grouped by subject on shelves within the bookcases — these represent directories and subdirectories. When used normally, the bookcases are effectively transparent and the books are located or stored depending on their subject matter. However, there may be times when more storage is needed or new subjects created and whole bookcases are moved or cleared. In these cases, the books are referred to using the bookcase as the reference — rather than subject matter.

The same can occur within Linux. Normally, access is via the file system, but there are times when it is easier to access the data as complete physical units rather than lots of files and directories. Hard disk copying and the allocation of part of the logical file system to a floppy disk are two examples of when access via the special */dev* file is used. Needless to say, accessing hard disks directly without using the file system can be extremely dangerous: the data is simply accessed by block numbers without any reference to the type of data that it contains. It is all too easy to destroy the file system and the information it contains. Another important difference between the access methods is that direct access can be performed at any time and with the mass storage in any state. To access data via the logical file system, data structures must be present to control the file structure. If these are not present, logical access is impossible.

Building the file system

When a Linux system is powered up, its system software boots the Linux kernel into existence. One of the first jobs performed is the allocation of mass storage to the logical file system. This process is called mounting and its reverse, the de-allocation of mass storage, is called unmounting. The *mount* command specifies the special file which represents the physical storage and allocates it to a target directory. When *mount* is complete, the file system on the physical storage media has been added to the logical file system. If the disk does not have a filing system, i.e. the data control structures previously mentioned do not exist, the disk cannot be successfully mounted.

The *mount* and *umount* commands can be used to access removable media, such as floppy disks, via the logical file system.

The disk is mounted, the data accessed as needed and the disk unmounted before physically removing it. All that is needed for this access to take place is the name of the special device file and the target directory. The target directory normally used is */mnt* but the special device file name varies from system to system. The *mount* facility is not normally available to end users for reasons that will become apparent later in this chapter.

The file system

Files are stored by allocating sufficient blocks of storage to contain all the data they contain. The minimum amount of storage that can be allocated is determined by the block size, which can range from 512 bytes to 8 kbytes in more recent systems. The larger block size reduces the amount of control data that is needed — but can increase the storage wastage. A file with 1,025 bytes would need two 1,024 byte blocks to contain it, leaving 1,023 bytes allocated and therefore not accessible to store other files. End of file markers indicate where the file actually ends within a block. Blocks are controlled and allocated by a superblock, which contains an inode allocated to each file, directory, subdirectory or special file. The inode describes the file and where it is located.

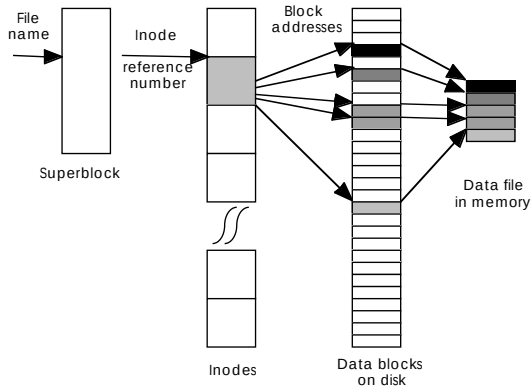
di_mod	File type, flags, access permission
di_nlink	Number of inode directory references
di_uid	File owner user id
di_gid	File owner group id
di_size	File size
di_addr	13 address fields for data block allocation
.	
.	
.	
.	
.	
.	
.	
di_addr	
di_atim	
di_mtime	
di_ctime	

The inode structure

Using the library and book analogy, the *superblock* represents the library catalogue which is used to determine the size and location of each book. Each book has an entry — an *inode* — within the catalogue.

The example *inode* below, which is taken from a Motorola System V/68 computer, contains information describing the file type, status flags and access permissions (read, write and execute) for the three classifications of users that may need the file: the owner who created the file originally, any member of the owner's group and, finally, anyone else. The owner and groups are identified by their identity numbers, which are included in the *inode*. The total file size is followed by 13 address fields, which point to the blocks that have been used to store the file data. The first ten point directly to a block, while the other three point indirectly to

other blocks to effectively increase the number of blocks that can be allocated and ultimately the file size. This concept of direct and indirect pointers is analogous to a library catalogue system: the *inode* represents the reference card for each book or file. It would have sufficient space to describe exactly where the book was located, but if the entry referred to a collection, the original card may not be able to describe all the books and overflow cards would be needed. The *inode* uses indirect addresses to point to other data structures and solve the overflow problem.



File access mechanism

Why go to these lengths when all that is needed is the location of the starting block and storage of the data in consecutive blocks? This method reduces the amount of data needed to locate a complete file, irrespective of the number of blocks the file uses. However, it does rely on the blocks being available in *contiguous* groups, where the blocks are consecutively ordered. This does not cause a problem when the operating system is first used, and all the files are usually stored in sequence, but as files are created and deleted, the free blocks become fragmented and intermingled with existing files. Such a fragmented disk may have 20 Mbytes of storage free, but would be unable to create files greater than the largest contiguous number of blocks — which could be 10 or 20 times smaller. There is little more frustrating than being told there is insufficient storage available when the same system reports that there are many megabytes free. Linux is more efficient in using the mass storage — at the expense of a more complicated directory control structure. For most users, this complexity is hidden from view and is not relevant to their use of the file system.

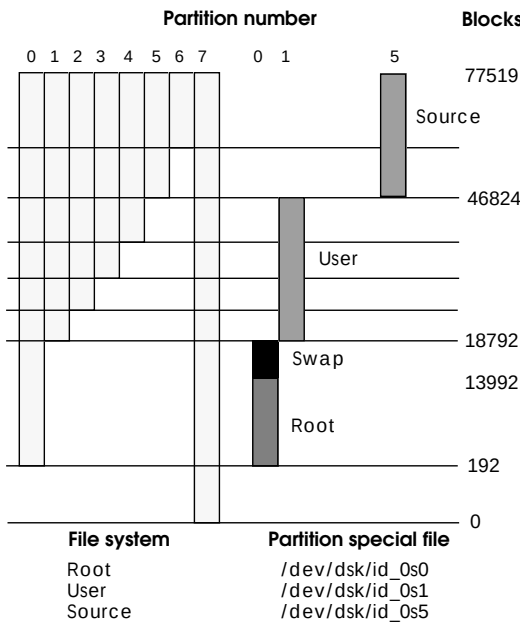
So what actually happens when a user wants a file or executes a command? In both cases, the mechanism is very similar. The operating system takes the file or command name and looks it up within the *superblock* to discover the *inode* reference number. This is used to locate the *inode* itself and check the access permissions before allowing the process to continue. If permission is granted, the *inode* block addresses are used to locate the data blocks stored on hard disk. These blocks are put into memory to reconstitute the

file or command program. If the data file represents a command, control is then passed to it, and the command executed.

The time taken to perform file access is inevitably dependant on the speed of the hard disk and the time it takes to access each individual block. If the blocks are consecutive or close to each other, the total access time is much quicker than if they are dispersed throughout the disk. Linux also uses mass storage as a replacement for system memory by using memory management techniques and its system response is therefore highly dependant on hard disk performance. UNIX uses two techniques to help improve performance: partitioning and data caching.

Disk partitioning

The concept of disk partitioning is simple: the closer the blocks of data are to each other, the quicker they can be accessed. The potential distance apart is dependant on the number of blocks the disk can store, and thus its storage capacity. Given two hard disks with the same access time, the drive with the largest storage will give the slowest performance, on average. The principle is similar to that encountered when shopping in a small or large supermarket. It takes longer to walk around the larger shop than the smaller one to fetch the same goods.

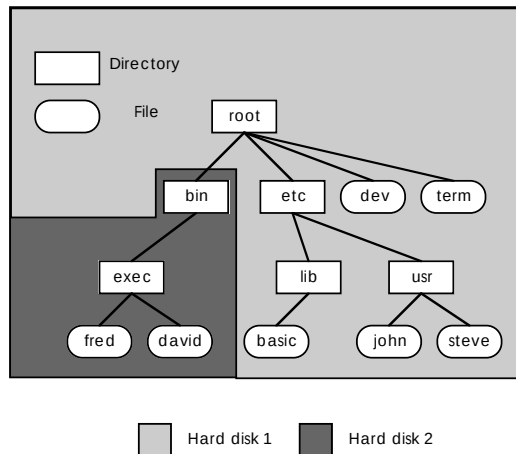


Example Linux partitioning

Linux has the option of partitioning large hard disks so the system sees them as a set of smaller disks. This greatly reduces the amount of searching required and considerably improves overall access times. Each partition (or slice, as it sometimes called) is created by allocating a consecutive number of blocks to it. The partition is treated exactly as if it is a separate mass storage device

and can be formatted, have a file system installed and mounted, if required. The partitions can be arranged so they either overlap or are totally separate. In both cases, an installed file system cannot exceed the partition size (i.e. the number of blocks allocated to it) and the lower boundaries of the file system and partition are the same. With non-overlapped partitions, the file system cannot be changed so it overlaps and destroys the data of an adjacent partition. With an overlapped arrangement, this is possible. Changing partition dimensions requires, at best, the reinstallation of the operating system or other software and, at worst, may need them to be completely rebuilt. Fortunately, this only usually concerns the system administrator who looks after the system. Users do not need to worry about these potential problems.

The Motorola System V/68 implementation uses such techniques as shown. The standard hard disk has 77,519 physical 512 byte blocks, which are allocated to 8 overlapping partitions. The whole disk can be accessed using partition 7, the lower 192 blocks of which are reserved for the boot software, which starts UNIX when the system is powered on. Partition 0 has root, the main file system, installed in blocks 192 to 13,991. Blocks 13,992 to 18,791 are used as a swap area for the virtual memory implementation and do not have a file system as such. Partition 1 is used to implement a User file system as far as block 46,823. This could not have been implemented using partitions 2, 3 or 4 without creating a gap — and effectively losing storage space. A third file system, Source, is implemented in partition 5 to use the remaining blocks for data storage.



The file system and physical storage

Partitioning provides several other advantages. It allows partitions to be used exclusively by Linux or another operating system, such as MS-DOS, and it reduces the amount of data backup needed to maintain a system's integrity. Most Linux implementations running on an IBM PC allocate partitions to either MS-DOS or Linux and the sizes of these partitions are

usually decided when the Linux software is first installed. Some implementations use the same idea, but create large MS-DOS files, which are used as UNIX partitions. In both cases, partitioning effectively divides the single physical hard disk into several smaller logical ones and allows the relatively easy transfer from Linux to MS-DOS, and vice versa. The same principles are also used if Linux is running on an Apple PowerMAC as well.

Most tape backup systems access hard disks directly and not through the file system, so whole disks can be quickly backed up onto tape. In practice, it is common for only parts of the file system to require backing up, such as user files and data, and this is more efficient if the backup process is restricted to these specific parts of the file system. This is easily done using partitions which are used by different parts of the file system. To back up specific parts, the special */dev* file for that partition is used. With the file structure shown below, copying partition 2 to tape would back up all the files and subdirectories in the */root/bin* directory. Copying partition 1 would copy everything excluding the */root/bin* directory.

The Linux disk partitioning

The Linux operating system uses partitions to allow it to co-exist with MS-DOS files and disks as used on IBM PCs. Any IBM PC disk can be partitioned using the MS-DOS FDISK command to create separate partitions. These partitions can then be assigned to Linux and thus support both MS-DOS (and Windows) as well as Linux. The partition naming follows a simple syntax as shown below. In addition, Linux can also directly read and write to MS-DOS disks.

This ability allows MS-DOS disks and thus files to co-exist within a Linux system without the need for special utilities to mount MS-DOS hard and floppy disks and then transfer files from MS-DOS and Linux and vice versa.

Name	Description
<i>/dev/fd0</i>	The first floppy disk drive (A:).
<i>/dev/fd1</i>	The second floppy disk drive (B:).
<i>/dev/hda</i>	The whole first disk drive (IDE or BIOS compatible disk drive, e.g. ESDI, ST506 and so on).
<i>/dev/hda1</i>	The first primary partition on the first drive.
<i>/dev/hda2</i>	The second primary partition on the first drive.
<i>/dev/hda3</i>	The third primary partition on the first drive.
<i>/dev/hda4</i>	The fourth primary partition on the first drive.
<i>/dev/hdb</i>	The whole second disk drive.
<i>/dev/hdb1</i>	The first primary partition on the second drive.
<i>/dev/hdb2</i>	The second primary partition on the second drive.
<i>/dev/hdb3</i>	The third primary partition on the second drive.
<i>/dev/hdb4</i>	The fourth primary partition on the second drive.
<i>/dev/hdc</i>	The whole third disk drive.
<i>/dev/hdc1</i>	The first primary partition on the third drive.
<i>/dev/hdc2</i>	The second primary partition on the third drive.

/dev/hdc3	The third primary partition on the third drive.
/dev/hdc4	The fourth primary partition on the third drive.
/dev/hdd	The <i>whole</i> fourth disk drive.
/dev/hdd1	The first primary partition on the fourth drive.
/dev/hdd2	The second primary partition on the fourth drive.
/dev/hdd3	The third primary partition on the fourth drive.
/dev/hdd4	The fourth primary partition on the fourth drive.
/dev/sda	The <i>whole</i> first disk drive (LUN 0) on the first SCSI controller.
/dev/sda1	The first primary partition on the first drive.

The */proc* file system

Linux has an additional special file system called */proc*. This is not a file system in the true sense of the term but a simple method of getting information about the system using the normal tools and utilities. As will be shown, this is not always the case and there is at least one special utility commonly used with this file system. It is useful in making sure that all the drivers and other system components you expected to be installed are actually there. To access the */proc* file system, it must be built into the kernel. Most, if not all, standard Linux kernels do this to provide debugging information at the very least.

Data caching

One method of increasing the speed of disk access is to keep copies of the most recently used data in memory so it can be fetched without having to keep accessing the slower electro-mechanical disk. The first time the data is needed, it is read from disk and is copied into the cache memory. The next time this data is required, it comes directly from cache memory — without using the disk. This access can be up to 1,000 times faster — which greatly improves system performance. The amount of improvement depends on the amount of cache memory present and the quantity of data needed from disk. If cache memory exceeds the required amount of data, the maximum performance improvement is gained — all the data is read once and can be completely stored in cache memory. If the amount of data is larger than the amount of cache memory, that the actual disk has been updated. The system frequently caches the new data — so the *only* copy is in cache memory. As this memory is volatile, if the machine is switched off the data is lost. If this information also includes superblock and inode changes, the file system will have been corrupted and, at best, parts of it will have been destroyed. At worst, the whole file system can be lost by switching the power off without executing a power down sequence. Most times, an accidental loss of power will not cause any real damage — but it is playing Russian roulette with the system.

The user can force the system to update the disk by executing the sync command as required. This is a well recommended practice.

Multi-tasking systems

Most operating systems used on PCs today, such as MS-DOS, can only execute one application at a time. This means that only one user can use the computer at any time, with the further limitation that only one application can run at a time. While a spreadsheet is executing, the PC can only wait for commands and data from the keyboard. This is a great waste of computer power because the PC could be executing other programs or applications or, alternately, allow other users to run their software on it. The ability to support multiple users running multiple applications is called multi-user multi-tasking. This is a feature of Linux — and is one of the reasons for its rapid adoption. The multi-tasking techniques are where standard Linux falls down in that they use a time slice mechanism (as explained earlier in this chapter) and this is not real-time. As a result, the initial use of Linux into the embedded market has been restricted because of this and the amount of resources such as memory that it needs to function. This has prompted the development of embedded Linux (eLinux) that will be explained later in this chapter.

Multi-user systems

Given a multi-tasking operating system, it is easy to create a multi-user environment, where several users can share the same computer. This is done by taking the special interface program that provides the command line and prompts, and running multiple copies of it as separate processes. When a user logs into the computer, a copy of the program is automatically started. In the UNIX environment, this is called the shell, and there are several different versions available. The advantages of multi-user systems are obvious — powerful computer systems can be shared between several users, rather than each having a separate system. With a shared system, it can also be easier to control access and data, which may be important for large work groups.

With any multi-user system, it is important to prevent users from corrupting each others work, or gaining access to sensitive data. To facilitate this, Linux allocates each user a password protected login name, which uniquely identifies him. Each user is normally allocated his own directory within the file system and can configure his part of the system as needed. Users can be organised into groups and every file within the system is given access permissions controlling which user or group can read, write or execute it. When a file is accessed, the requesting user's identity (or ID) is checked against that of the file. If it matches, the associated permissions are checked against the request. The file may be defined as read only, in which case a request to modify it would not be allowed — even if the request came from the user who created it in the first place. If the user ID does not match, the group IDs are checked. If these match, the group permissions are used to judge the validity of the request. If neither IDs match, a

third set of permissions, known as others, are checked as the final part of this process.

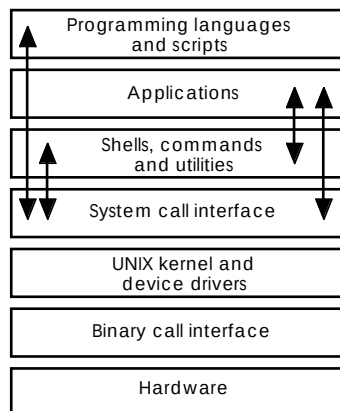
These permissions can be changed as required by the system administrator, who must set up the Linux system and control how much or how little access each user has to the system and its facilities. This special user (or superuser) has unlimited access by being able to assume any user and/or group identity. This allows an organised structure to be easily implemented and controlled.

Linux software structure

The software structure used within UNIX is very modular, despite its long development history. It consists of several layers starting with programming languages, command scripts and applications, shells and utilities, which provide the interface software or application the user sees. In most cases, the only difference between the three types of software is their interaction and end product, although the more naive user may not perceive this.

The most common software used is the shell, with its commands and utilities. The shell is used to provide the basic login facilities and system control. It also enables the user to select application software, load it and then transfer control to it. Some of its commands are built into the software but most exist as applications, which are treated by the system in the same way as a database or other specialised application.

Programming languages, such as C and Fortran, and their related tools are also applications but are used to write new software to run on the system.



Example Linux software structure

As shown in the diagram, all these layers of software interface with the rest of the operating system via the system call interface. This provides a set of standard commands and services for the software above it and enables their access to the hardware, filing systems, terminals and processor. To read data from a file, a set of system calls are carried out which locate the file, open it and transfer the required data to the application needing it. To find out

the time, another call is used, and so on. Having transferred the data, system calls are used to call the UNIX kernel and special software modules, known as device drivers, which actually perform the work required.

Up to this point, the software is essentially working in a standard software environment, where the actual hardware configuration (processor, memory, peripherals and so on) is still hidden. The hardware dependant software which actually drives the hardware is located at the binary call interface.

Of all the layers, the kernel is the heart of the operating system and it is here that the multi-tasking and multi-user aspects of Linux and memory control are implemented. Control is achieved by allocating a finite amount of processor time to each process — an application, a user shell, and so on.

When a process starts executing, it will either be stopped involuntarily (when its CPU time has been consumed) or, if it is waiting for another service to complete, such as a disk access. The next process is loaded and its CPU time starts. The scheduler decides which process executes next, depending on how much CPU time a process needs, although the priority can be changed by the user. It should be noted that often the major difference between UNIX variants and/or implementations is the scheduling algorithm used.

Processes and standard I/O

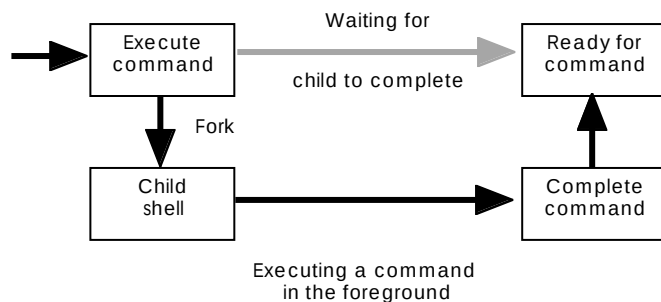
One problem facing multi-user operating systems is that of access to I/O devices, such as printers and terminals. Probably the easiest method for the operating system is to address all peripherals by unique names and force application software to directly name them. The disadvantage of this for the application is that it could be difficult to find out which peripheral each user is using, especially if a user may login via a number of different terminals. It would be far easier for the application to use some generic name and let the operating system translate these logical names to physical devices. This is the approach taken by Linux.

Processes have three standard files associated with them: `stdin`, `stdout` and `stderr`. These are the default input, output and error message files. Other files or devices can be reassigned to these files to either receive or provide data for the active process. A list of all files in the current directory can be displayed on the terminal by typing `ls<cr>` because the terminal is automatically assigned to be `stdout`. To send the same data to a file, `ls > filelist<cr>` is entered instead. The extra `> filelist` redirects the output of the `ls` command. In the first case, `ls` uses the normal `stdout` file that is assigned to the terminal and the directory information appears on the terminal screen. In the second example, `stdout` is temporarily assigned to the file `filelist`. Nothing is sent to the terminal screen — the data is stored within the file instead.

Data can be fed from one process directly into another using a 'pipe'. A process to display a file on the screen can be piped into another process which converts all lower case characters to upper case. It can then pipe its data output into another process, which pages it automatically before displaying it. This command line can be written as a data file or 'shell script', which provides the commands to the user interface or shell. Shell scripts form a programming language in their own right and allow complex commands to be constructed from simple ones. If the user does not like the particular way a process presents information, a shell script can be written which executes the process, edits and reformats the data and then presents it. There are two commonly used shell interfaces: the standard Bourne shell and the 'C' shell. Many application programs provide their own shell which hide the Linux operating system completely from the user.

Executing commands

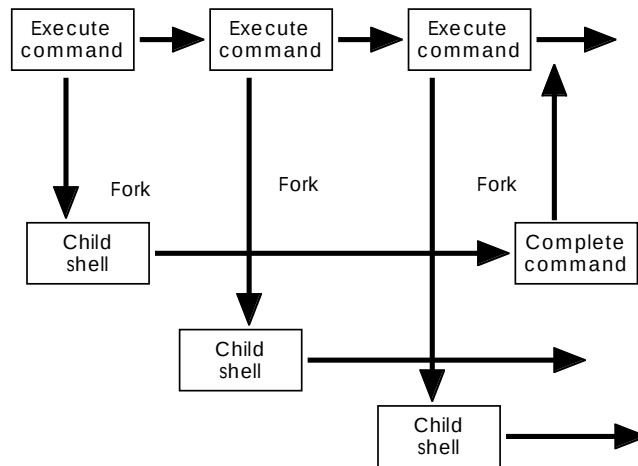
After a user has logged onto the system, Linux starts to run the shell program by assigning stdin to the terminal keyboard and stdout to the terminal display. It then prints a prompt and waits for a command to be entered. The shell takes the command line and deciphers it into a command with options, file names and so on. It then looks in a few standard directories to find the right program to execute or, if the full path name is used, goes the directory specified and finds the file. If the file cannot be found, the shell returns an error message.



Foreground execution

The next stage appears to be a little strange. The shell forks i.e. it creates a duplicate of itself, with all its attributes. The only difference between this new child process and its parent is the value returned from the operating system service that performs the creation. The parent simply waits for the child process to complete. The child starts to run the command and, on its completion, it uses the operating system exit call to tell the kernel of its demise. It then dies. The parent is woken up, it issues another prompt, and the mechanism repeats. Running programs in this way is called executing in the foreground and while the child is performing the required task, no other commands will be executed (although they will still be accepted from the keyboard!).

An alternative way of proceeding is to execute the command in background. This is done by adding an ampersand to the end of the command. The shell forks, as before, with each command as it is entered — but instead of waiting for the child to complete, it prompts for the next command. The example below shows three commands executing in background. As they complete, their output appears on stdout. (To prevent this disrupting a command running in foreground, it is usual to redirect stdout to a file.) Each child is a process and is scheduled by the kernel, as necessary.



Background execution

Physical I/O

There are two classes of physical I/O devices and their names describe the method used to collect and send data to them. All are accessed by reading and writing to special files, usually located in the */dev* directory, as previously explained. Block devices transfer data in multiples of the system block size. This method is frequently used for disks and tapes. The other method is called character I/O and is used for devices such as terminals and printers.

Block devices use memory buffers and pools to store data. These buffers are searched first to see if the required data is present, rather than going to the slow disk or tape to fetch it. This gives some interesting user characteristics. The first is that frequently used data is often fetched from memory buffers rather than disk, making the system response apparently much better. This effect is easily seen when an application is started a second time. On its first execution, the system had to fetch it in from disk but now the information is somewhere in memory and can be accessed virtually instantaneously. The second characteristic is the performance improvement often seen when the system RAM is increased. With more system memory, more buffers are available and the system spends less time accessing the disk.

It is possible to access some block devices directly, without using memory buffers. These are called raw accesses and are used, for example, when copying disks.

For devices such as terminals and printers, block transfers do not make much sense and here character transfers are used. Although they do not use memory buffers and pools in the same way as block devices, the Linux kernel buffers requests for single characters. This explains why commands must be terminated with a carriage return — to tell the kernel that the command line is now complete and can be sent to the shell.

Memory management

With many users running many different processes, there is a great potential for problems concerning memory allocation and the protection of users from accesses to memory and peripherals that are currently being used by other users of the system. This is especially true when software is being tested which may attempt to access memory that does not exist or is already being used. To solve this sort of problem, Linux depends on a memory management unit (MMU) — hardware which divides all the memory and peripherals into sections which are marked as read only, read or write, operating system accesses only, and so on. If a program tries to write to a read only section, an error occurs which the kernel can handle. In addition, the MMU can translate the memory addresses given by the processor to a different address in memory.

Linux limitations

Unfortunately, Linux is not the utopian operating system for all applications. Its need for memory management, many megabytes of RAM and large mass storage (>40 Mbytes) immediately limits the range of hardware platforms it can successfully run on. Mass storage is not only used for holding file data. It also provides, via its virtual operating system and memory management scheme, overflow storage for applications which are too big to fit in the system RAM all at once. Its use of a non-real-time scheduler, which gives no guarantee as to when a task will complete, further excludes UNIX from many applications.

Through its use of memory management to protect its resources, the simple method of writing an application task which drives a peripheral directly via its physical memory is rendered almost impossible. Physical memory can be accessed via the slow `‘/dev/mem’` file technique or by incorporating a shared memory driver, but these techniques are either very slow or restrictive. There is no straightforward method of using or accessing the system interrupts and this forces the user to adopt polling techniques.

In addition, there can be considerable overheads in managing all the look up tables and checking access rights etc. These overheads appear on loading a task, during any memory

allocation and when any virtual memory system needs to swap memory blocks out to disk. The required software support is usually performed by an operating system. In the latter case, if system memory is very small compared with the virtual memory size and application, the memory management driver will consume a lot of processing power and time in simply moving data to and from the disk. In extreme cases, this overhead starts to dominate the system — which is working hard but achieving very little. The addition of more memory relieves the need to swap and releases more of the processing power to execute the application.

Finally, the system makes extensive use of disk caching techniques, which use RAM buffers to hold recent data in memory for faster access. This helps to reduce the system performance degradation, particularly when used with a combination of external page swapping and slow mass storage. The system does not write data immediately to disk but stores it in a buffer. If a power failure occurs, the data may only be memory resident and is therefore lost. As this can include directory structures from the superblock, it can corrupt or destroy files, directories or even entire systems! Such systems cannot be treated with the contempt other, more resilient, operating systems can tolerate — Linux systems have to be carefully started, shut down, administered and backed up.

One of the more interesting things about the whole Linux movement is that give the developers a problem and someone somewhere will find a way round the problem and come up with a new version. Given that Linux in its initial form is not ideal for embedded systems, can an embedded real-time version be created that would allow the wealth of Linux software to be executed and reused? The answer has been yes.

eLinux

While UNIX is a wonderful operating system for workstations, desktops and servers, it suffers from several restrictions that prevented it from being used in the embedded system environment. It was large in terms of memory requirements both as main memory and virtual memory where hard disk storage is used to extend the amount of memory the system thinks it has. It makes a lot of assumptions about its environment that may not be true in an embedded system. How many embedded systems do you see with terminals and hard disks? These problems can be overcome but the real issue is the characteristics shown by the kernel. Yes the kernel is multi-tasking and yes it will support multiple users if needed but it does not support real-time operation. As a result, the embedded system community has largely excluded it from consideration. There were also problems with access to code, licensing, royalties that didn't help its cause either.

Linux then appears and as it developed under the various licences that required easy access to the source code, it started to

do several things. Firstly, it attracted a large applications base who ended up writing applications and drivers that were freely available. At this point several people started to play with the kernel and its internals. Access was as simple as downloading the code from a web site and the idea of a version to support embedded designers started to come together.

The first thing that has to be remembered that while most real-time applications are embedded systems, not all embedded systems need real-time support. Taking this one step further, many designs actually need the ability to complete critical operations within a certain time frame and need a “fast enough” system and not necessarily a real-time one. This is important as the processing power available to designers today is getting faster and faster and this means that the need to chase every last bit of processing power from the system is no longer needed and instead a faster processor and memory can be used. In this case, the margin between the time taken to complete the operation and when it needs to be completed is so great that there is no longer any need to have a real-time system. This has led to the introduction of non-real-time operating systems with fast processors as a viable alternative for some embedded system designs. So the mantra of “it is embedded and therefore must need a real-time operating system” has been shattered. It is no longer such a universal truth and many designs can use non-real-time operating systems for an embedded design providing the software is designed to cope with the characteristics that the system provides. Indeed the last chapter goes through a design that does exactly that and uses MS-DOS with no multitasking or real-time capabilities as a real-time data logger.

This change coupled with the growth and confidence in Linux-based applications and systems has encouraged the use of Linux in embedded design. This has meant that embedded Linux has taken two development directions: the first concerns adapting the operating system to fit in a constrained hardware system with reduced amounts of memory and peripherals. This is based on stripping out as much of the software that can be done while maintaining the required functionality and this has dramatically reduced the amount of memory that is needed to run the operating system. Add to that support for RAM disks and other solid state memory technology such as flash and an ability to boot up system tasks without the need for a terminal connection and these basic problems are addressed. However, the next issue is not so easy and is concerned with modifying the Linux kernel to provide real-time support.

The standard kernel is multi-tasking and uses a sophisticated fairness scheme that will try and give a fair distribution and sharing of the processing time to the different tasks and threads that are running at the time. It is not pre-emptive in that a currently running task or thread cannot be shut down and replaced as soon as a higher priority task needs to run. So how can Linux be made to run real-time? There are three main methods.

1. Run the standard kernel on a fast enough hardware. This can achieve some impressive figures for task switching (60 microseconds is not uncommon) but it should be remembered that a fast task switch does not mean that there is not some thread or task in the system that might block the kernel and prevent the system from working. This approach requires making some risk assessments over the potential for this to happen. It is fair to say that Linux drivers and code tends to be consistently written following the recommended practices and that this gives a high level of confidence. However, in the same way that MS-DOS can be used for real-time embedded systems by carefully designing how the software is written and runs, the same can be done for Linux without changing the kernel.

Care needs to be taken however to ensure that there are no hidden problems and in particular tasks and drivers that can hog the execution time and block operations. As drivers are normally supplied in source code, they can be inspected (and modified if necessary) to prevent this. This does require some level of expertise and understanding of how the driver was written, which can be a daunting prospect at first.

2. Replace the standard kernel with a real-time version. This is a little strange in that one of the reasons why Linux is popular is because of the stability of its kernel and the fact that it is freely available. Yet this route advocates replacing the kernel with a real-time version. Several proposals have been made and have included the idea of using a standard RTOS and wrapping it so that it looks like a normal Linux kernel from a software perspective.
3. Enhance the standard kernel with pre-emptive scheduling. In this case, the standard kernel is modified to allow blocking tasks to be pre-empted and removed from execution.

Now the joy of working with Linux is that there are many keen developers ready to meet the challenge but this can lead to many different implementations. It is fair to say that there are several types of embedded Linux implementations available now that use different techniques to provide embedded support.

One method that is used with the TLinux/ TAI releases is to use a second kernel that has real-time characteristics and thus the operating system becomes a hybrid with normal Linux software running under the Linux kernel and real-time tasks running under the second real-time kernel. Communication between the Linux and RTOS worlds is performed using shared memory. This does work but is not as elegant as some purists would like in that why have the Linux kernel there in the first place and also it forces the software developer to classify software into the two camps. It is a suitable solution for very time critical applications where the

Linux components are not critical and are restricted to housekeeping and other activities.

The alternative to a second kernel is to make the kernel real-time itself. It turns out that there is real-time support deep down in the kernel e.g. the `SCHED_FIFO` and `SCHED_` calls that support up to 100 priority levels. The idea is that these priority levels are serviced first and then if there are no such tasks that need execution then control can be passed to normal Linux tasks and threads via the standard scheduler. This provides a priority scheme but there are some restrictions. While the calls are present in the interfaces, this does not mean that the implementations actually support or enforce them. In addition, there is no pre-emption which means that a lower priority task or thread can still prevent a higher priority one from pre-empting and gaining control. However, providing the implementation does support these schemes, this can provide an improved real-time Linux environment. Couple it with fast hardware and good interrupt latencies can be obtained.

So ideally, a pre-emptive version of the kernel is needed. It turns out that the standard SMP (Symmetric Multi Processing) version of Linux does just that and by modifying it slightly to support a single processor, it can become a transparent priority-based pre-emptive eLinux kernel. This is the approach that Monta Vista has taken with its eLinux support.

In summary, the restrictions that prevented eLinux from becoming a mainstream embedded RTOS have by and large been removed and eLinux is poised to become a dominant player in the RTOS market.